Computers & Security 148 (2025) 104160

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

NTLFlowLyzer: Towards generating an intrusion detection dataset and intruders behavior profiling through network and transport layers traffic analysis and pattern extraction

MohammadMoein Shafi [a,*], Arash Habibi Lashkari [a,b], Arousha Haghighian Roudsari [c]

[a] Department of Electrical Engineering and Computer Science, York University, Toronto, Ontario, Canada
[b] Behaviour-Centric Cybersecurity Center (BCCC), School of Information Technology, York University, Toronto, Ontario, Canada
[c] School of Computing, Gachon University, Songnam, South Korea

ARTICLE INFO

Keywords:
Network security
Behavioral profiling
Zero-day attacks
Anomaly detection
Feature selection
Behavior similarity
Pattern extraction
NTLFlowLyzer
BCCC-CIC-IDS2017
Taxonomy of profiling

ABSTRACT

Network security remains a critical concern in modern computing systems due to the constant emergence of threats and attacks. This paper introduces a comprehensive behavioral profiling solution to address the limitations of current intrusion detection methods in identifying zero-day attacks and novel malicious behaviors. Beginning with raw network data, the proposed framework progresses through multiple stages, ultimately culminating in the creation of activity-specific profiles. Central to this approach is NTLFlowLyzer, a novel network traffic analyzer, which generates an updated dataset, BCCC-CIC-IDS2017, for enhanced profile generation. The core of the profiling system leverages the distinct behaviors exhibited by individual features and the diverse correlations observed across various activities. The profiling procedure attains accuracy and robustness by integrating a novel feature selection algorithm and a pattern extraction process. Furthermore, behavior similarity is introduced to quantify the resemblance between activities based on their features and behaviors. We rigorously evaluate the effectiveness of our model by subjecting it to comprehensive testing, followed by meticulous comparison with previous works. Our proposed framework proficiently characterizes eight malicious activities with an accuracy rate surpassing 99.8%, while displaying promising performance in profiling various other activities. These findings, derived from our comprehensive experiments, provide valuable guidance for accurately implementing behavioral profiling.

1. Introduction

Modern computing systems face constant exposure to diverse threats and attacks, underscoring the significance of network security. Monitoring and analyzing network activities to detect anomalous or malicious behaviors are pivotal for safeguarding networks and their assets. However, prevailing intrusion detection systems struggle to identify zero-day attacks and malicious activities. Behavioral profiling, which entails modeling normal behavior for each network entity (users, devices, etc.) and detecting deviations or anomalies, emerges as a technique to address this challenge (Shafi et al., 2024a).

Nonetheless, crafting accurate and resilient behavioral profiles presents considerable challenges. Unique activities may necessitate distinct profiling approaches due to their diverse behaviors. This necessitates a flexible and comprehensive framework to create behavioral profiles for varied network activities systematically (Shafi et al., 2024b).

This paper introduces a holistic solution with raw network data and culminates activity-specific profiles. We select the CIC-IDS2017 dataset, but due to limitations, we propose NTLFlowLyzer, a novel network traffic analyzer, to generate an updated dataset, BCCC-CIC-IDS2017 (BCCC-Dataset, 2024), for profile creation.

The profiling core is founded on two principles: (1) each feature exhibits distinctive behavior in activities, and (2) features display diverse correlations across activities. By amalgamating these concepts, we devise profiles capturing activity essence. Our novel feature selection algorithm and pattern extraction integrate these principles. We also introduce behavior similarity, quantifying activity resemblance based on features and behaviors.

This paper presents groundbreaking contributions to network behavior profiling:

- A novel feature selection algorithm.
- A behavior similarity calculation algorithm.

* Corresponding author.
E-mail address: moeinsh@yorku.ca (M. Shafi).

https://doi.org/10.1016/j.cose.2024.104160
Received 11 August 2023; Received in revised form 15 November 2023; Accepted 10 October 2024
Available online 19 October 2024

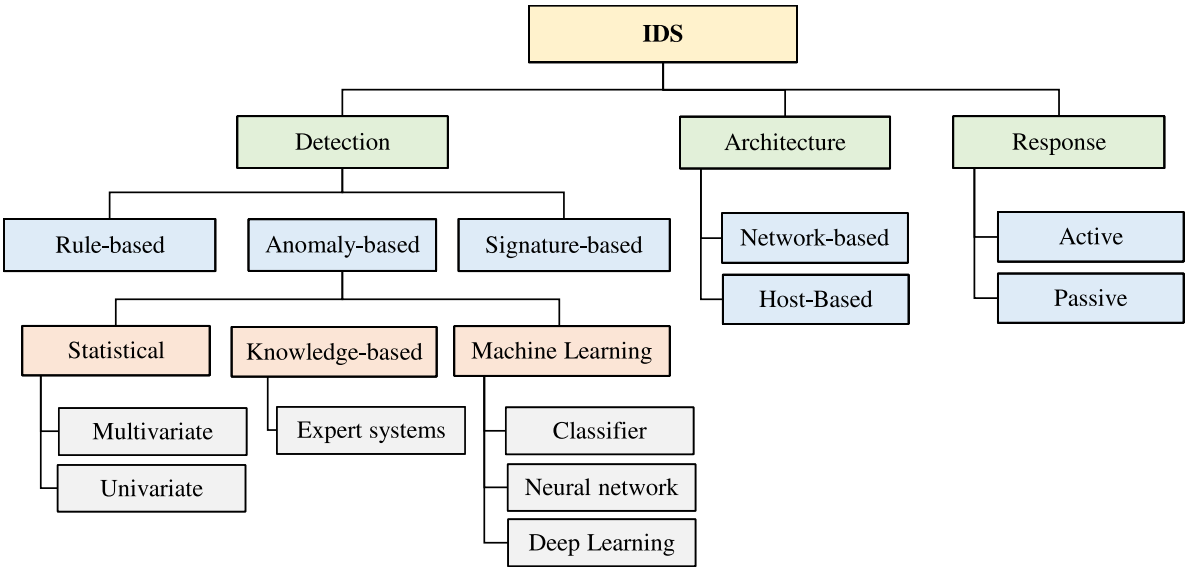


Fig. 1. Taxonomy of IDS detection methods.

- A new network traffic analyzer, NTLFlowLyzer.
- A benchmark dataset, BCCC-CIC-IDS2017 (BCCC-Dataset, 2024).
- A novel profiling system.
- A taxonomy of profiling techniques.

The rest of the paper is organized: Section 2 provides related work context. Section 3 details our profiling model, including feature selection, behavior similarity calculation, and profile definition. Section 4 presents NTLFlowLyzer and its comparison with CICFlowMeter. Section 5 outlines the CIC dataset and its relevance. Section 6 describes the experimental setup and results using our model. Section 7 analyzes the findings, and Section 8 concludes the paper and outlines future directions.

2. Literature review

This section comprehensively reviews Intrusion Detection Systems (IDS) techniques. Fig. 1 illustrates an organized taxonomy of IDS methods. Moreover, recent influential studies are categorized by methodology.

Limitations of previous research are delineated, highlighting areas for our contributions. Our paper addresses these gaps, proposing profiling as the optimal solution, substantiated by the profiling techniques' background (Fig. 2). Recent distinguished works utilizing profiling are methodically categorized, enabling precise comparison. Identified shortcomings in these studies drive our enhancements.

2.1. Intrusion detection systems

This section reviews prior work on intrusion detection methods. Firstly, an IDS taxonomy and key concepts are discussed. The synthesis then highlights existing literature limitations and indicates areas our proposed model addresses.

IDS are categorized by architecture, response, and detection methods (Fig. 1). Architecturally, IDS can be host-based, utilizing host audit data like system calls, or network-based, working with network traffic data (Li et al., 2019). Host-based offers deep insights but can strain resources; network-based is easier to deploy but struggles with encrypted traffic (Ayyagari et al., 2021).

In terms of response, IDS is passive or active. Passive monitors, while active, intervenes upon detection (Kocher and Kumar, 2021). Detection methods span signature-based (misuse), rule-based, and anomaly-based (behavior) IDS.

2.1.1. Signature-based IDS

Signature-based detection relies on existing attack signatures. It matches network traffic to known patterns, flagging matches as intrusions. Regular updates to the dataset are essential for detecting new attacks, often done manually or via dedicated websites. (AlYousef and Abdelmajeed, 2019) proposed auto-updating signatures; (Li et al., 2019) introduced a blockchain IDS with incremental signature updates.

Escalating network traffic challenges traditional IDS hardware, prompting cloud-based solutions. Cloud usage raises privacy concerns. (Wang et al., 2018) suggested a privacy-preserving IDS based on fog devices. Rapid attack evolution weakens signature-based methods' effectiveness. While efficient for known attacks, signature-based methods falter against new, unrecorded patterns (Kocher and Kumar, 2021; Dina and Manivannan, 2021). Detecting new attacks is pivotal, given the upsurge in novel attack types and malware variants.

2.1.2. Rule-based IDS

Rule-based detection employs If-Then and If-Else-Then rules to spot targeted attacks, relying on prior attack knowledge. (Monzer et al., 2022) proposed a complex cyber–physical system IDS using a physical model yet overlooked model uncertainties. (Sonchack et al., 2015) emphasized robust rule sets, suggesting adaptation for local network traffic optimization. Manual collaboration expense is a concern. (Sagala, 2015) employed a honeypot-based IDS, transferring data to activate snort rules.

(Tomandl et al., 2014) developed VANET IDS for fake message detection through neighboring vehicle data. (Afzal and Lindskog, 2016) advocated rule-based ICS IDS, focusing on an optimized ruleset for evolving attacks. These models automate rule generation for specific limited scenarios yet fail against zero-day or unknown attacks. As attacks increase, intricate rules strain software and hardware processing.

2.1.3. Anomaly-based IDS

Anomaly-based detection models normal network behavior. User or system patterns from past network use serve as the baseline. Deviations from this norm flag anomalies as potential attacks (Kocher and Kumar, 2021; Liao et al., 2013). Vital for detecting unknown attacks, these models produce higher false positives than signature-based methods (Mushtaq et al., 2022; Dina and Manivannan, 2021). Anomaly-based IDS comprises knowledge-based, statistical-based, and machine-learning-based approaches.

Statistics-based methods analyze data records, forming a statistical model of typical behavior. Metrics like standard deviation, mean,

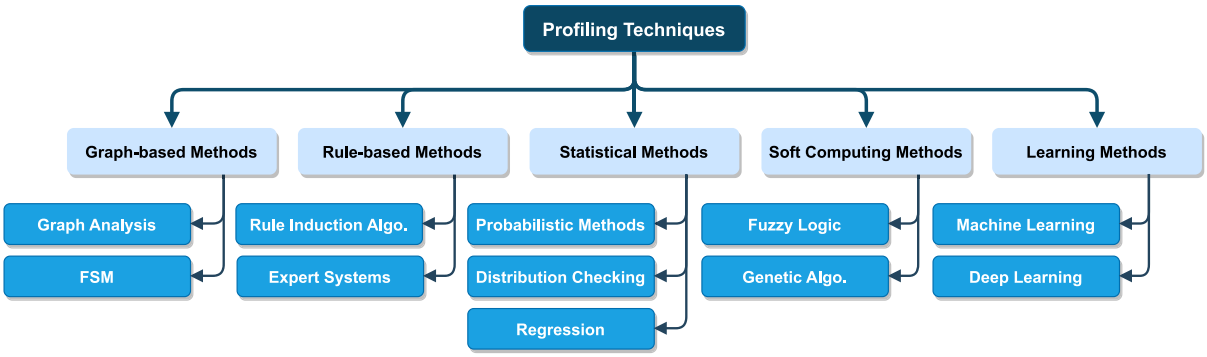


Fig. 2. Profiling techniques taxonomy.

and mode informs this model. Univariate monitors individual metrics; multivariate assesses relationships among variables. Multivariate high-dimensional data distribution estimation is a challenge. Knowledge-based utilizes expert systems akin to rule-based models, recognizing actions and network traffic using existing system data. Machine learning methods enhance pattern-matching via adaptable, robust training data use, rendering them popular choices for anomaly-based IDS (Khraisat et al., 2019).

Machine learning-based IDSs encompass session-based, packet-based, or flow-based features. Yet, real-time intrusion detection poses challenges. (Kim and Pak, 2022) propose deferred decision-making with a hybrid classifier, optimizing detection speed and accuracy. Complexity is a concern. (Qiu et al., 2022) integrate Dempster–Shafer theory for flow and packet-based early attack detection.

Data volume hinders intrusion detection efficiency. (Herrera-Semenets et al., 2022) reduce data through label generation, relabeling, and duplicate removal. (Baldini and Amerini, 2022) apply Morphological Fractal Dimension in a sliding window for anomaly detection. (Asif et al., 2021) use MapReduce for big dataset processing. Deep Learning excels in intrusion detection. (Imran et al., 2022) propose a Deep Learning-based IDS achieving 99.65% accuracy. (Liu et al., 2022) utilize auto-encoder, MLP, and clustering for attack detection. (Mushtaq et al., 2022) use auto-encoders and LSTM. (Ravi et al., 2022) employ recurrent models for 99% accuracy. (Li et al., 2022) use clustering to label attacks.

Anomaly-based detection, like other methods, has merits and limitations. Ongoing research strives for better accuracy and reduced false positives and negatives.

2.1.4. Synthesis

Previous method limitations include:

1. Signature-based IDS excel with known attacks but struggle against unknown or zero-day attacks.
2. Rule-based IDS are intricate, effective for known attacks, but incapable of detecting unknown ones.
3. Anomaly-based IDS have lower accuracy and higher false positives than signature-based IDS, but detect unknown attacks.
4. Profiling for reduced false positives in anomaly-based IDS is complex.
5. Network-based IDS grapple with vast data handling.
6. Real-time detection is crucial for a reliable IDS.

This paper adopts a profiling model to address issues 1, 2, 5, and 6.

2.2. Behavioral profiling

Behavioral profiling is crucial in network security, identifying individual or group behavior patterns for normal or anomalous activity detection. Five primary categories compose behavioral profiling techniques: Learning Methods, Soft Computing Methods, Statistical Methods, Rule-based Methods, and Graph-based Methods. Fig. 2 illustrates the taxonomy, following detail each category and its families.

Learning Methods predict behavior using past data, comprising Machine Learning and Deep Learning. Machine Learning learns patterns and predicts using algorithms (Janiesch et al., 2021). Deep Learning employs deep neural networks for complex pattern extraction (Sarker, 2021).

Soft Computing Methods use fuzzy logic, neural networks, and genetic algorithms for uncertain data handling (Cui et al., 2019). Fuzzy Logic captures uncertainty with truth degrees, while Genetic Algorithms optimize solutions (Ibrahim, 2016), (Meng and Saddeh, 2020).

Statistical Methods analyze behavior patterns with Regression, Probabilistic, and Distribution Checking Methods. Regression models variables' relationships (Kolascyk, 2013). Probabilistic applies randomness for solutions. Distribution Checking examines data distribution (Silva et al., 2022).

Rule-Based Methods use predefined rules for classification. Rule Induction Algorithms learn from data; Expert Systems rely on human knowledge (Abdulganiyu et al., 2023), (Fürnkranz, 2013).

Graph-based Methods model data as a graph, analyzing relationships. Valuable for complex data and anomaly identification (Pourhabibi et al., 2020), (Khoshraftar and An, 2022).

Subsequent subsections explore prior work, technique analysis, and strengths/weaknesses.

2.2.1. Previous IDS behavioral profiling research

This section offers an encompassing overview of recent behavioral profiling research in network security, classified by employed approaches as outlined earlier. Proposed models, experiments, and analyses within each category are demonstrated. Limitations of prior works are summarized, emphasizing coverage by our proposed model.

- **Rule-based Works**
  (Kapetanakis et al., 2014) adopt a case-based reasoning approach, profiling attackers' characteristics during an attack to differentiate known profiles from a pool of attack data and human infiltration behaviors.
- **Learning-based Works**
  (Hou et al., 2022) present a hierarchical attention network-based flow-vector generation approach for identifying malicious traffic, utilizing network profiling and machine learning. (Singh et al., 2015) propose an intrusion detection method using Online Sequential Extreme Learning Machine, incorporating feature selection strategies and profile reduction for scalability.
- **Statistical-based Works**
  (Muraleedharan et al., 2010) utilize IP flow characteristics and chi-square detection to propose a flow-based system for anomaly detection, establishing normal behavior models and detecting threats by analyzing network traffic behavior.

- **Mixed Works**
  - **Learning and Soft Computing Methods**
    (Rabbani et al., 2020) enhance cloud service provider behavior modeling through combined learning and self-optimized machine learning using a Probabilistic Neural Network. Particle Swarm Optimization boosts system performance, enabling user behavior categorization and attack recognition.
  - **Learning and Statistical Methods**
    (Herrmann et al., 2013) utilize DNS resolver data for tracking algorithms. Behavioral traits inform profiles, pattern extraction, and session matching. Test instance classification employs the closest comparable training case and Euclidean distance.

2.2.2. Synthesis

There are several shortcomings identified with the existing methods, including:

1. Working only on specific types of malicious behavior: Almost all of the previous profiling works focus on specific known signatures or patterns, limiting their ability to detect novel threats.
2. Lack of comprehensive feature set: Previous research indicates that comprehensive feature sets are crucial for accurate behavioral profiling (von Ziegler et al., 2021). However, almost all of the previous works worked with a dataset containing less than 50 features.
3. Low accuracy: Several studies have pointed out the accuracy limitations in conventional intrusion detection systems (Hou et al., 2022; Singh et al., 2015; Rabbani et al., 2020; Muraleedharan et al., 2010).
4. Easy to circumvent in case of knowing the IDS: Existing systems can be vulnerable to evasion techniques (Herrmann et al., 2013; Muraleedharan et al., 2010; Hou et al., 2022).
5. Lack of good data and model (profile) visualization: The importance of data and model visualization has been emphasized in (Vellido, 2020; Unwin, 2020; Midway, 2020).
6. Lack of a defined profile per attack or malware: Almost all of the previous profiling works lack a defined clear profile per benign and malicious activity.
7. Not suitable for online detection: Online detection challenges are discussed in (Alrawashdeh and Purdy, 2016; Aljanabi et al., 2021; Hsu et al., 2019), however, almost all of the previous profiling works suffer from this shortcoming.
8. Not applicable on profiling the zero-day attacks and vulnerabilities: In response to the ever-evolving threat landscape, the demand for a profiling system equipped to effectively profile zero-day attacks becomes imperative (Ahmad et al., 2023; Guo, 2022; Barros et al., 2022).
9. Lack of Scalability with the new applications, protocols, and features: To address the challenges stemming from the introduction of new applications, protocols, and features, scalability becomes a critical necessity (Shaikh et al., 2009; Khan et al., 2019; Mighan and Kahani, 2021).
10. Lack of a comprehensive dataset: To meet the requirements for effective training and analysis, a comprehensive dataset is vital, ensuring the reliability and thoroughness of the process (). Nonetheless, as detailed in Section 7.7.1, datasets used in prior research exhibit numerous issues.
11. High Complexity in terms of time, preparation, real-time detection, and performance: Most prior research overlooked the complexity challenge in their model designs.
12. Lack of adaptable variance per profile (dynamic threshold): Dynamic profiling systems are essential for adapting to and effectively responding to the evolving threat landscape. In the ever-changing world of cybersecurity, threats and attack techniques are constantly evolving and becoming more sophisticated. Static or rigid profiling systems are unable to keep up with this pace of change and quickly become obsolete, leaving organizations vulnerable to emerging threats. (He et al., 2015; Liang et al., 2019; Lin et al., 2019).
13. Not suitable for encrypted traffic detection: The increasing prevalence of encryption in modern network communication and the security monitoring challenges it presents necessitate a profiling system suitable for encrypted traffic detection (Wang et al., 2022; Garcia et al., 2021).

In this study, we have effectively addressed and resolved the first nine above-mentioned issues.

3. Proposed model

This section introduces our innovative model for profiling network traffic to derive distinct behavior-based profiles for each activity. Addressing previous limitations, our approach offers a novel perspective on network traffic analysis. The overarching architecture of our solution is visualized in Fig. 3, providing an initial glimpse into our novel framework. Subsequent subsections will delve into specific facets of our model, discussing model input and feature extraction, feature selection, behavior similarity metrics, the profiling core, and profile-assigning. Additionally, the components of the proposed model are delineated in detail through the presentation of Algorithms 1, 2, and 3, offering a comprehensive understanding of our innovative approach to network traffic analysis and behavior profiling.

3.1. Feature extraction

Our solution begins by extracting crucial insights from raw network traffic data using our specialized analyzer. These extracted features form the foundation of our subsequent analyses and profiling. In the upcoming subsections, we explore how this analyzed data and these features are harnessed to extract the patterns and further create profiles. In Section 4 we elaborate on our analyzer and the extracted features in detail, while in Section 5 we delve into the details of the selected raw network data and further the prepared analyzed data.

3.2. Feature selection

This section introduces a novel approach for activity-specific feature selection. This approach operates on datasets comprising n instances and m features, ultimately providing the most suitable feature set for each label. The methodology leverages a correlation graph, mapping features to nodes and quantifying their relationships. Our process involves creating a fully connected graph, initializing with zero edge weights, and updating these weights based on feature correlations. This results in a weighted graph $G = (V, E)$ with nodes (V) and weighted edges (E) reflecting correlations.

For each activity label $a \in A$, the algorithm systematically calculates correlations between feature pairs within instances labeled as a and updates edge weights in graph G accordingly. Subsequently, edges with weights below a predefined threshold are eliminated. The algorithm then identifies the strongest path within graph G, ensuring a node is visited just once throughout this process. The nodes, representing features, within this strongest path are ultimately utilized as the optimal feature set for the given activity a.

Unique representations emerge for each activity as we repeat the process for each label in the dataset. Fig. 6 demonstrates distinct correlation graphs for various activities, highlighting the need to tailor features to specific profiles. This contrasts with conventional uniform feature sets for all activities. Optimal feature sets are identified by finding the strongest path of a specified length within the correlation graph. Algorithm 1 outlines this procedure.

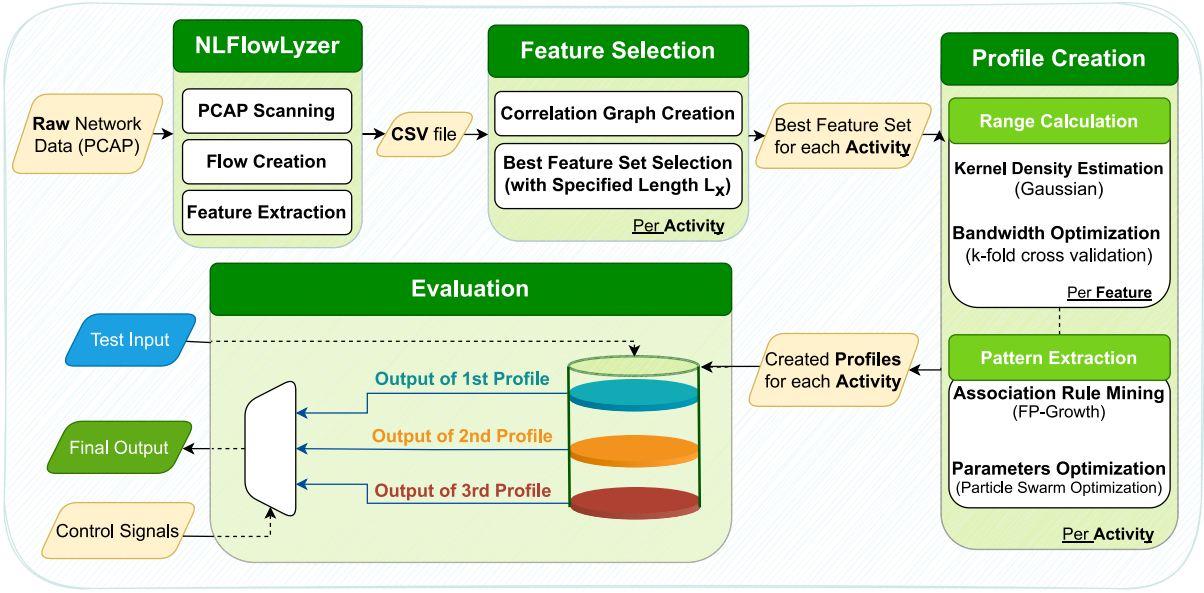


Fig. 3. General solution.

Common correlation algorithms like Pearson's, Kendall's, and Spearman's were tested for network security context (Cohen et al., 2009; Abdi, 2007; Myers and Sirois, 2004). Selection depends on variable relationships and research questions (Chok, 2010; Bolboaca and Jäntschi, 2006). Experiments compared these algorithms (Section 6, Fig. 7).

The introduction of this new feature selection methodology is driven by the imperative need for feature selection algorithms to align seamlessly with our profiling algorithm. Given that our profiling definition inherently encompasses the possibilities and correlations among feature values, we recognized the necessity to devise an approach that can effectively address this unique challenge within the profiling framework. In essence, the concept of this feature selection algorithm evolved organically during the design and implementation of the profiling algorithm.

It is noteworthy that the existing literature encompasses feature selection algorithms that consider feature correlations. In their respective studies, (Zhou et al., 2022; Wang et al., 2020; Akhiat et al., 2021; Potharaju and Sreedevi, 2018; Zhang and Hancock, 2011) present feature selection methods that prioritize the identification of features highly correlated with the target variable, while minimizing inter-feature correlations. (Zhou et al., 2022) leverage mutual information, (Wang et al., 2020) utilize a graph-based Laplacian matrix, (Akhiat et al., 2021) employ a correlation-based filtering approach, (Potharaju and Sreedevi, 2018) opt for a low-dimensional embedding approach, and (Zhang and Hancock, 2011) apply a graph-based regularization approach. However, none of these methods explicitly address the intricate "chain of correlations" among features. Our innovative approach, utilizing a graph-based framework, unlocks the potential to assist the profiling algorithm in extracting patterns associated with the complex possibilities of feature values.

The Pattern Extraction subsection (Section 3.4.2) underscores the algorithm's integration into our profiling system and pattern extraction module. We discuss its pivotal role, deviating from conventional methods, and enhancing our comprehensive profiling methodology's effectiveness and robustness.

3.3. Behavior similarity

We introduce the novel concept of behavior similarity, a metric designed to quantify the likeness between different network activities. This metric draws from the earlier-discussed feature selection graph concept.

Algorithm 1 Feature Selection Algorithm

Input: Dataset with n instances and m features
Output: Best feature set for each activity label
Initialize a fully connected graph $G = (V, E)$ with $V = f_1, f_2, \ldots, f_m$ and $E = (f_i, f_j) | 1 \leq i, j \leq m$ with initial edge weights of zero
for each activity $a \in A$ **do**
    Calculate the correlation between each pair of features for instances with label a
    Update the edge weights in graph G based on the calculated correlations
    Remove all edges with weights below a defined threshold
    Find the strongest path with a defined length in the graph G without visiting any node twice
    The nodes (features) in the strongest path are used as the best feature set for activity a
end for
Return Best feature set for each activity

Leveraging the constructed feature selection graphs for all activities, our approach yields a robust formula, Eq. (1), to assess the similarity between activities A_1 and A_2 denoted as $S(A_1, A_2)$. The formula updates similarity values by analyzing edges E_1 and E_2 connecting features f_i and f_j in both graphs.

The procedure initiates with a similarity value of zero for activities A_1 and A_2. The similarity measure is systematically adjusted by examining each pair of features and corresponding edges and utilizing an intricate calculation mechanism. The process considers the presence, direction, and magnitude of edges. Normalizing the outcome by the number of correlations results in a floating-point value between $(-1, 1)$. Higher values signify greater similarity between behavioral patterns, aiding assessments of activity likeness and potential risks.

This algorithm yields pivotal benefits across multiple domains. It offers precise similarity quantification, enhancing network anomaly detection, intrusion identification, and risk assessment. High similarity indicates related activities, enabling proactive risk mitigation. Comparing activities against benign and malicious behaviors strengthens profiling and categorization. The behavior similarity algorithm augments network analysis, providing organizations with enhanced comprehension and strategic decision-making capabilities for intricate network activities.

$$S(A_1, A_2) = \sum_{f_i, f_j}^{Features} \begin{cases} 1 & \text{if } E_1 > 0 \text{ and } E_2 > 0 \\ 1 & \text{if } E_1 < 0 \text{ and } E_2 < 0 \\ -1 & \text{if } E_1 > 0 \text{ and } E_2 < 0 \\ -1 & \text{if } E_1 < 0 \text{ and } E_2 > 0 \\ 1 & \text{if } \neg E_1 \text{ and } \neg E_2 \\ -1 & \text{if } \neg E_1 \text{ and } E_2 \\ -1 & \text{if } E_1 \text{ and } \neg E_2 \end{cases} \quad (1)$$

3.4. Profiling core

Several critical steps are employed to achieve the utmost precision and efficiency in a profile definition, as outlined in Algorithm 2. Our proposed profiling model is founded on two fundamental concepts. The first revolves around the unique behavioral characteristics exhibited by each feature across different activities. The second concept pertains to the diverse correlations observed among various features across distinct activities. Building upon the best features identified in the preceding subsections, the subsequent sections provide a detailed elucidation of our profile creation algorithm centered around these two primary principles.

In the Range Calculation subsection, we address each selected feature within each activity, effectively transforming the infinite float value space into finite categories. The ensuing Profiling Core subsection delves into a comprehensive exploration of the possibilities encompassing all these ranges and features within each activity. Finally, in the last subsection, we demonstrate the process of assigning profiles to new inputs based on the patterns extracted earlier. This pivotal step completes the profiling process, facilitating identifying and recognition of novel activities based on their similarities to existing profiles.

3.4.1. Range calculation

Following the selection of features for each activity, the subsequent step involves constructing the profile of each activity based on the two fundamental concepts as mentioned earlier. This subsection focuses on the first concept, which posits that each feature exhibits distinctive behavior across various activities. For example, as illustrated in Fig. 8, the average duration value varies significantly between benign and DDoS activities. As substantiated in Section 6, we firmly believe that each feature value within each activity conforms to certain characteristic ranges. To identify these ranges, we employ the Kernel Density Estimation (KDE) (Chen, 2017) function, defined as:

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right) \quad (2)$$

Here, $\hat{f}_h(x)$ represents the kernel density estimate of the probability density function at the point x, utilizing the chosen kernel function with bandwidth h. n denotes the total number of data points in the dataset under consideration, x_i represents the ith data point, and K function refers to the kernel function, defining the shape of the kernel. The bandwidth h, also known as the smoothing parameter, is responsible for controlling the width of the kernel, significantly influencing the smoothness of the resultant density estimation (Raykar and Duraiswami, 2006).

In this study, we utilize the Gaussian function (Keerthi and Lin, 2003) as our kernel:

$$K(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right) \quad (3)$$

Consequently, the KDE function with Gaussian as the kernel is expressed as:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(x - x_i)^2}{2h^2}\right) \quad (4)$$

The Gaussian kernel function is widely adopted for Kernel Density Estimation due to its symmetric and smooth properties, facilitating accurate density estimation (Węglarczyk, 2018).

To determine the optimal bandwidth (h) for our KDE estimation, we employed k-fold cross-validation (Rodriguez et al., 2009). For each fold, we calculated the mean integrated squared error (MISE) (Marron and Wand, 1992) by evaluating the discrepancy between the estimated density function ($\hat{f}_{-i}(x)$) with the i-th observation omitted and the true underlying probability density function ($f(x)$) of the data. The MISE for the i-th fold is expressed as:

$$MISE_i(h) = \int [\hat{f}_{-i}(x) - f(x)]^2 dx \quad (5)$$

Subsequently, the cross-validation score ($CV(h)$), representing the average MISE over all n data points, was computed as follows:

$$CV(h) = \frac{1}{n} \sum_{i=1}^{n} MISE_i(h) \quad (6)$$

The bandwidth h_{opt} that minimized the cross-validation score was then selected as the optimal bandwidth for our KDE estimation. This critical selection ensures an optimal trade-off between capturing fine data details and generating a smooth and accurate density estimate.

After determining the bandwidth, we assigned a range to each local maximum, considering the local maximum point as the center of the corresponding range. We identified all the ranges for each feature in a specific activity by following this approach. We effectively reduced dependence on the dataset by mapping each record to its nearest center range within each activity and its corresponding features. This method allows us to identify the primary regions of the data, thereby mitigating the impact of duplicate, unbalanced, or noisy data as we focus on the local maxima.

Importantly, it is imperative to clarify that *"range_1"* for *"feature_1"* in *"activity_1"* may differ from *"range_1"* for *"feature_1"* in *"activity_2"*. For instance, let us consider the feature named *"pkts_count"*, which can exhibit three ranges in benign activity but four in DDoS activity. This distinction arises from the divergent nature of these activities, warranting separate consideration. Consequently, the values for *"range_1"* concerning the *"pkts_count"* feature in the benign activity would differ from those in the DDoS activity. The general procedure for identifying data ranges is illustrated in the following algorithm:

3.4.2. Behavior profiling by pattern extraction

In this subsection, we culminate our behavior profile creation by incorporating the second concept elucidated earlier. The second idea pertains to the diverse correlations observed among different features across various activities. As an illustrative example, let us consider the features "duration" and "packet count". In benign activities, these two features exhibit no meaningful correlation. In contrast, in an SSH Patator activity, the "packet count" demonstrates a high positive correlation with "duration", as visually depicted in Fig. 6. This subsection delves into the intricacies of applying these feature correlations to the previously explained ranges. In essence, we seek to determine the associations between different ranges within one feature and within another. To elaborate, when the value of "duration" falls within "range1", what are the plausible values for "packet count"? Notably, these new correlations, or rather, possibilities, may differ for each activity compared to another. The pattern extraction process herein unveils these latent behavioral nuances unique to each activity.

A crucial consideration is that our novel feature selection algorithm thoughtfully identifies the most correlated features. Consequently, this stage of profile creation invariably yields meaningful and rational patterns. This underscores the criticality of our feature selection algorithm, for it ensures that the pattern extraction process is anchored in the underlying correlations between the features' values. Indeed, the successful identification of patterns hinges on the existence of substantial correlations between the features. As such, if the selected features lack

meaningful correlations due to common algorithms like information gain, the ensuing pattern extraction process likely yields no meaningful patterns based on the features' value variations.

We employ Association Rule Mining algorithms to extract the diverse possibilities between different feature values within an activity; we employ Association Rule Mining algorithms (Kotsiantis and Kanellopoulos, 2006). This family of algorithms aligns seamlessly with our second concept, as previously discussed. After a meticulous evaluation of various algorithms within this family (Luna et al., 2019), we have chosen to utilize the FP-Growth algorithm (Shawkat et al., 2022) due to its superior performance compared to other alternatives (Singh et al., 2014). However, this algorithm requires the selection of two crucial parameters, namely minimum support and confidence ratios.

The minimum support ratio represents the minimum proportion of transactions required for a pattern to be considered. In contrast, the minimum confidence ratio denotes the minimum proportion of transactions that must satisfy the pattern to be deemed valid (Shawkat et al., 2022). In our pursuit of identifying the optimal values for these parameters, we adopted a rigorous approach involving the utilization of optimization algorithms. After thoughtful consideration, we determined that the Particle Swarm Optimization (PSO) algorithm (Kennedy and Eberhart, 1995) is best suited for this purpose.

The choice of PSO over other optimization techniques was based on several factors. Firstly, PSO's ability to effectively navigate complex search spaces aligns well with the challenges posed by the parameter optimization task (Zhang et al., 2015). Secondly, PSO is known for its inherent parallelism, which allows for efficient computation and reduced processing time (Zeng et al., 2020). Finally, PSO's adaptive nature enables it to dynamically adjust its search strategy based on the evolving fitness landscape, ensuring convergence towards optimal solutions (Jensi and Jiji, 2016; Wang and Song, 2019).

Several comparative studies have demonstrated the superior performance of PSO over other metaheuristic algorithms in various optimization tasks. (Tharwat and Schenck, 2021) evaluated PSO against Genetic Algorithm (GA), Differential Evolution (DE), and Artificial Bee Colony (ABC) algorithms and found that PSO outperformed the others in terms of solution quality and convergence speed for problems with complex search spaces. (Pratama and Suyanto, 2020) compared PSO, GA, and DE for feature selection in classification tasks and found that PSO achieved superior classification accuracy. (Kachitvichyanukul, 2012) investigated the performance of PSO, GA, and DE in optimizing Support Vector Machine (SVM) parameters and found that PSO consistently outperformed the others in terms of classification accuracy and model complexity. (Khan and Sahai, 2012) compared PSO, GA, and DE for neural network training and found that PSO achieved faster convergence and superior generalization performance. These studies highlight PSO's effectiveness in handling multidimensional, non-linear optimization problems, its inherent parallelism and adaptive nature, and its superior performance compared to other metaheuristic algorithms.

This nature-inspired algorithm maintains a swarm of particles, with each particle representing a potential solution. The PSO algorithm guides these particles towards the most promising regions of the search space. The particle's movement is influenced by its current position, its best previously attained position, and the overall best position discovered by the swarm (Kennedy and Eberhart, 1995). Mathematically, the position update for each particle i at iteration $t+1$ can be expressed as follows:

$$\mathbf{v}_i^{(t+1)} = w \cdot \mathbf{v}_i^{(t)} + c_1 \cdot r_1 \cdot (\mathbf{p}_i - \mathbf{x}_i^{(t)}) + c_2 \cdot r_2 \cdot (\mathbf{g}^{(t)} - \mathbf{x}_i^{(t)}) \tag{7}$$

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)} \tag{8}$$

where $\mathbf{x}_i^{(t)}$ represents the position of particle i at iteration t, $\mathbf{v}_i^{(t)}$ denotes its velocity, $\mathbf{p}_i$ is the best position found by particle i so far, and $\mathbf{g}^{(t)}$ signifies the best position obtained by the entire swarm at iteration t. The parameters w, c_1, and c_2 regulate the impact of the particle's previous velocity, its individual experience, and the collective knowledge of the swarm, respectively. The random coefficients r_1 and r_2 introduce stochasticity to the particle's movement. Through iterative refinement, PSO effectively guides the swarm towards the optimal parameter values, leading to enhanced performance in our parameter tuning process (Poli et al., 2007).

The performance metric utilized to optimize the particles is based on maximizing accuracy while minimizing false positive rates. Thus, in each iteration of the algorithm, we calculate the value of the equation $Accuracy - False\_Positive$. The higher the value of this equation, the better the result. Consequently, the particles move in a manner that optimizes this equation, thereby improving both precision and recall.

Once the parameter tuning for a specific activity is completed, we run the algorithm one final time with the best parameters to extract the patterns of that activity. These extracted patterns form the core of the profile for each activity. Algorithm 2 outlines the different steps involved in profile creation. The patterns extracted during this stage play a pivotal role in the decision-making process during the profile-assigning stage.

Algorithm 2 Pattern Extraction Algorithm

Require: A dataset D with n activities, each represented by m categorical selected features.
Ensure: A set of activity-specific patterns for each activity.

Parameters: Initial confidence value C, Initial support value SU, Particle Swarm Optimization steps s
for each activity a in dataset D **do**
  Run Association Rule Mining Algorithm (FP-Growth) with initial C and SU
  Employ Particle Swarm Optimization (PSO) to fine-tune the values of C and SU
  for each step s in Particle Swarm Optimization **do**
    for each particle p in particles **do**
      Update C and SU based on step size s
      Calculate Accuracy and False Positive rates
      Update particle positions according to the value of $< Accuracy - FP >$
      Save the best C and SU for each particle P_i based on the highest $< Accuracy - FP >$ value
    end for
  end for
  Run the FP-Growth with the Best C and SU
end for
Termination:
return Activity-specific patterns for each activity

3.5. Profile-assigning

Having obtained the patterns for each activity, the next step involves profiling new activities using these sets of patterns. To achieve this, we analyze the ranges associated with the corresponding features of each activity and create a pattern for the test input with each activity. Consequently, we obtain one pattern for each activity. Subsequently, we compare the test input pattern with the previously created patterns for each activity. The profile-assigning procedure for the test input is as follows: If only one activity's pattern is matched, the new input is labeled with the corresponding activity's label. If more than one pattern is matched, and one of them corresponds to the benign activity, the new input is labeled as *"Suspicious"*. Otherwise, it is classified as an *"Attack"*. Lastly, if no pattern is found in any activities, the new input is labeled as *"Unknown"*. The overarching procedure for assigning the correct profile to new inputs is depicted in Algorithm 3.

Our novel approach to profiling network activity combines a novel graph-based feature selection technique with a novel pattern extraction module to deliver efficient results. The feature selection stage extracts the best feature set for each activity by finding the strongest paths in

Algorithm 3 Profile Assigning Algorithm

Require: n Activities with corresponding Patterns P, Each with m selected features, and the Test input data
Ensure: Profile of the test input
Range Calculation:
for each activity a in dataset D **do**
  for each $f_{i,j}$ where $0 \leq i < n$ and $0 \leq j < m$ **do**
    Calculate the range r associated with $f_{i,j}$
    Replace r as the value of $f_{i,j}$ for a
  end for
  Save the test pattern TP_a
end for
Match Patterns:
for each pattern p_a in P and each tp_a in TP **do**
  if $p_a == tp_a$ **then**
    Add activity a to possible profiles list PPL
  end if
end for
Profile Assignment:
if PPL size == 1 **then**
  Assign the corresponding profile
else
  if PPL size == 0 **then**
    Assign "***Unknown***"
  else if "Benign" in PPL **then**
    Assign "***Suspicious***"
  else
    Assign "***Attack***"
  end if
end if
return Profile of the test input

the feature's correlation graph. The range calculation module mines the underlying data behavior and the pattern extraction module generates the patterns by considering the correlations between previously defined ranges in different features. Finally, in the profiling-assigning stage, the corresponding profile for each new test input is calculated. In the experiment Section 6, we have implemented the profiling model and designed a test scenario, and in the analysis and discussion Section 7, we have analyzed our proposed model from different perspectives and discussed the possible future avenues.

This innovative and practical solution establishes a new standard for accurately profiling network behavior. Its applicability empowers organizations, including firewalls, IDS/IPS, Security Information and Event Management (SIEM), and Unified Threat Management (UTM) systems, to strengthen their network security and threat detection mechanisms, particularly in the context of detecting zero-day and previously unknown malicious activities.

4. Implementation

This section introduces NTLFlowLyzer (BCCC-NTLFlowLyzer, 2024), comparing it with one of the most popular existing tools, CICFlowMeter (Lashkari et al., 2017). The architecture, flow creation process, behavior selection, and feature extraction are discussed.

4.1. NlFlowLyzer: Network layer traffic feature extractor

In this work, we present the development of a new open-source tool for extracting network traffic flow features from pcap files named NTLFlowLyzer. The tool is in Python and primarily uses the DPKT library to read the pcap file. Our tool builds upon the previously developed CICFlowMeter tool in Java by (Lashkari et al., 2017), addressing issues identified in its implementation and theoretical underpinnings, such as incomplete features' formulas, unfinished flow creation, etc. Furthermore, NTLFlowLyzer enhances and expands upon CICFlowMeter by integrating a range of new features, as presented in Table 1. This integration provides an enriched capability for analyzing network flows and extracting valuable insights. The primary motivation for the development of NTLFlowLyzer was to overcome the limitations of CICFlowMeter and provide a more efficient and accurate tool for extracting valuable features from network traffic. In the following subsections, we will discuss the shortcomings of CICFlowMeter and the improvements implemented in NTLFlowLyzer, as well as provide a detailed overview of the tool's features and functionality.

4.1.1. CICFlowMeter covered shortcomings and issues

CICFlowMeter is a previous tool that was developed in 2017 using the Java programming language. While the tool effectively extracted features from network traffic, our work seeks to improve upon it by addressing issues identified in its implementation and theoretical foundations. The primary motivation for developing our tool, NTLFlowLyzer, is to provide a more efficient and accurate means of extracting valuable features from network traffic, overcoming the limitations of CICFlowMeter.

In this section, we will discuss the issues with CICFlowMeter in detail, and describe the improvements we made in our implementation. The following is a list of issues we identified with CICFlowMeter. For more information on how we addressed these issues please refer to the GitHub page of NTLFlowLyzer (BCCC-NTLFlowLyzer, 2024):

1. Incorrect flow creation
2. CICFlowMeter's low performance
3. Creating empty CSV files for certain pcap files
4. Empty flow list creation for attack pcap files
5. Time-consuming features for specific pcap files
6. Installation on Windows and Linux
7. Payload bytes of the first packet issue
8. Loading network interfaces issues on Debian 10
9. PSH flag features issue
10. Down/Up ratio feature issue
11. ICMP protocol issue
12. Negative values in IAT statistics issue
13. Handling large files issues
14. Manual labeling shortcoming
15. Pcapng extension shortcoming
16. ARP flows issue

In addressing these issues, we added new features to NTLFlowLyzer and corrected the calculation of some of the previous features. A comparison of these features is presented in Table 1.

NTLFlowLyzer leverages the Python programming language instead of Java, providing several advantages in this domain. Python's simpler and more concise syntax facilitates both writing and reading code, while its rich set of libraries and frameworks allows for the implementation of advanced features with ease. Moreover, Python is easier to use and learn than Java, which increases accessibility for researchers and practitioners in the field. Python's ease of use and readability also enable smoother development processes and easier maintenance and modification of code over time. These advantages make Python a popular choice for such applications, and NTLFlowLyzer benefits significantly from its flexibility and versatility.

4.1.2. Architecture

NTLFlowLyzer is designed with a flexible architecture that allows for effective extraction of network flows, improved speed through multi-thread functionality, and ease of development and modification. The architecture is based on the latest software development principles

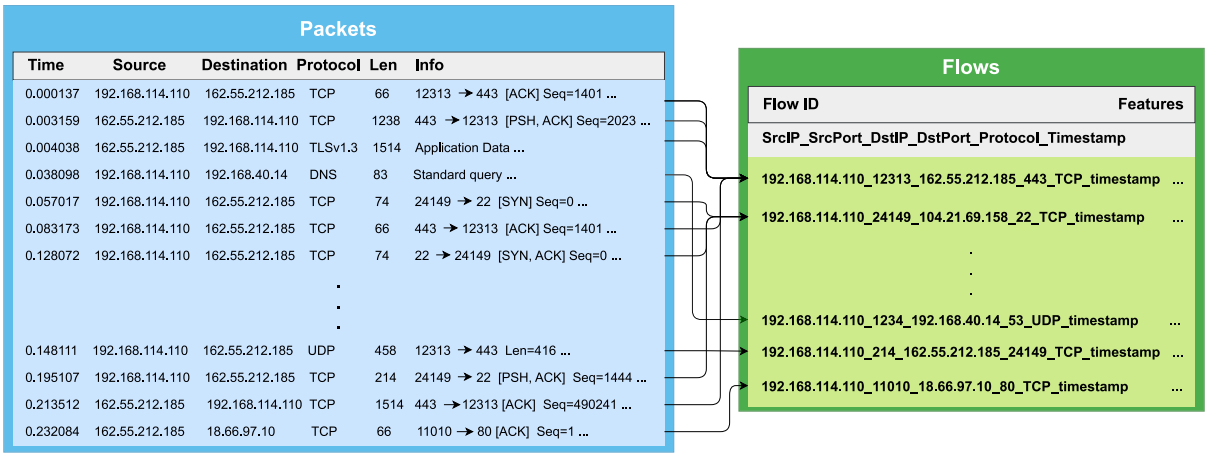


Fig. 4. Flow creation.

and design patterns, making it easy for other researchers to use and extend the tool for future work.

The tool architecture and details of each component are available on the GitHub page of the project (BCCC-NTLFlowLyzer, 2024). The user must provide the input network traffic pcap file and specify the desired output format. The tool then utilizes three main modules: FlowCapturer, FeatureExtractor, and Writer. The NetFlowCapturer module reads packets from the input file using the DPKT library and creates network flows. The FeatureExtractor module extracts relevant features from the output of the previous module. Finally, the Writer module prepares the output file containing the flows and their extracted features.

4.1.3. Flow creation

Our approach exhibits notable differences from previous works, particularly regarding flow definition and termination criteria. A network traffic flow is a sequence of packets with the same six attributes: source IP, source port, destination IP, destination port, protocol, and timestamp. Unlike other works, our approach considers bidirectional communication between a source and destination as a single flow. Additionally, we consider the timestamp of the packets, as it allows us to differentiate between communication (flows) that occurred at different times and potentially exhibited other behavior. For our discharges, we use the start time of the flow, which is determined by the timestamp of the first packet in the flow. Including a timestamp in the flow, the definition provides a more accurate representation of the communication taking place. It is worth noting that the general procedure of flow creation is visualized in Fig. 4, which highlights the steps involved in our method.

We also differ from previous works in terms of terminating a flow. To close a flow, our approach utilizes four distinct criteria, and the occurrence of any one of these criteria is sufficient for flow closure. These criteria include:

- Receipt of two FIN flags from sender and receiver;
- Presence of an RST flag;
- Flow duration exceeding the maximum threshold time;
- Flow idle time, defined as the time between packets being added to the flow, exceeding the maximum flow idle time.

These criteria have been carefully selected to ensure our flows accurately reflect the underlying network traffic. To the best of our knowledge, our approach represents a unique and novel approach to flow termination, as prior works have yet to consider all of these criteria.

4.1.4. Behavior selection and feature extraction

Prudent selection and extracting pertinent features are pivotal in crafting an efficacious network behavior analysis model. This section delineates our approach to behavior selection and feature extraction.

Fig. 5 elucidates our classification of six primary network behavior categories: Time-based, Flag-based, Rate-based, Header-based, Payload-based, and Side-based. Notably, behaviors not fitting the first five categories find inclusion in the final category, Side-behavior. This taxonomy emerges from a thorough literature review and empirical network traffic observations.

It is noteworthy that network behavior categorization can diverge based on the approach. Nonetheless, our core model remains constant, irrespective of categorization. For ease of comprehension, we offer a feature list in Table 1, contrasting NTLFlowLyzer and CICFlowMeter features.

For in-depth feature insight and precise definitions, the NTLFlowLyzer project's GitHub repository serves as a comprehensive resource.

5. Updated intrusion detection dataset (BCCC-CIC-IDS2017)

Effective evaluation of our proposed profiling model hinges on dependable datasets. This section scrutinizes prevalent public IDS datasets, identifying limitations. Our chosen dataset, CIC-IDS2017, offers an improved, realistic, and current portrayal of real-world network traffic, ideally suited for assessing our profiling model. Notably, we highlight disparities between our generated CSV file and the dataset's CICFlowMeter CSV file.

5.1. Available IDS traffic comparison

In this subsection, we undertake a comprehensive comparison of the most reputable IDS datasets to evaluate their suitability for our research objectives.

DARPA (Lincoln Laboratory 1998–99): This dataset includes diverse activities like FTP, email, browsing, and SNMP, along with artificially injected attacks such as buffer overflow and DoS. However, it lacks real-world traffic representation, newer attack types, and actual attack samples due to its age (McHugh, 2000; Brown et al., 2009)

KDD'99 (University of California, Irvine 1998–99): Derived from DARPA, KDD'99 integrates attacks like Smurf-Dos, but exhibits imbalanced attack class distribution and redundancy with its predecessor (Al Jallad et al., 2020; Tavallaee et al., 2009). NSL-KDD was constructed utilizing KDD (Tavallaee et al., 2009) to address some of the KDD's weaknesses (McHugh, 2000).

DEFCON (The Shmoo Group,2000–2002): DEFCON offers versions with intrusive activities like buffer overflows and port scanning. However, it lacks normal background traffic, reducing its applicability to real-world scenarios (Nehinbe, 2009).

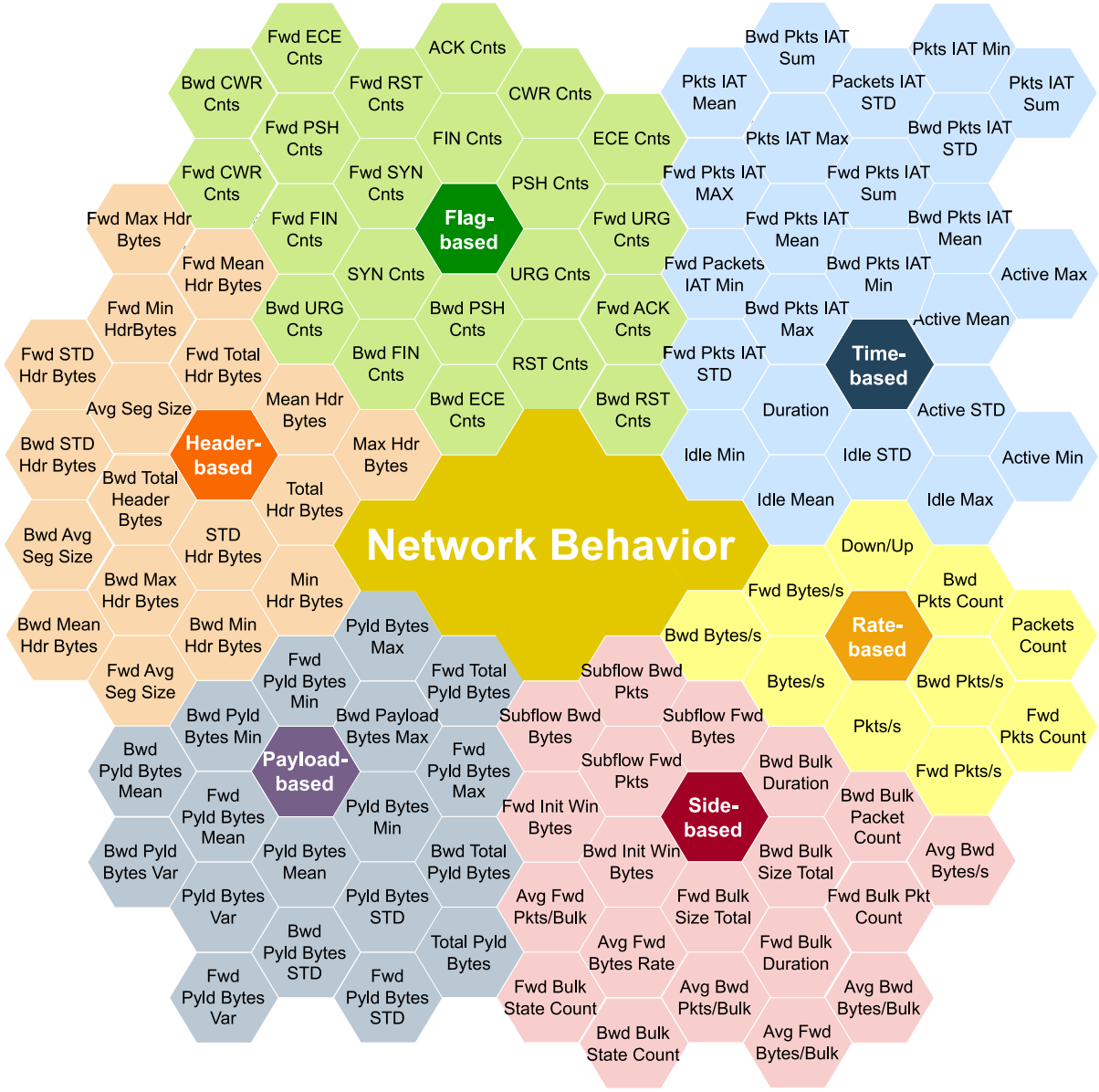


Fig. 5. Network behavior.

CAIDA (Center of Applied Internet Data Analysis 2002–2016): CAIDA presents datasets focused on specific attacks or events, potentially limiting their representativeness. Their anonymization approach masks protocol information and payload, impacting utility (Shiravi et al., 2012).

LBNL (Lawrence Berkeley National Laboratory and ICSI 2004–2005): LBNL offers full header traffic data from a medium-sized network, although privacy concerns led to payload exclusion. Despite its realism, it may lack the comprehensive attack diversity desired (Nechaev et al., 2004).

CDX (United States Military Academy 2009): CDX provides network traffic of essential services like Web and DNS, useful for testing alert rules. Nonetheless, its diversity and data volume are limited, potentially constraining its practicality (Sangster et al., 2009).

Kyoto (Kyoto University 2009): This dataset, generated using honeypots, features attacks against them. While offering useful features for IDS analysis, it may lack false positives, crucial for real-world alert minimization (Song et al., 2011; Sato et al., 2012; Chitrakar and Huang, 2012).

Twente (University of Twente 2009): Twente's dataset is collected from honeypot network traffic, providing labeled data. However, its attack type variety and data volume might not fully reflect real-world scenarios (Sperotto et al., 2009).

UMASS (University of Massachusetts 2011): UMASS dataset offers trace files of wireless applications and network packets. Its limitations include a lack of attack diversity and traffic volume for robust evaluation (Prusty et al., 2011).

ISCX2012 (University of New Brunswick 2012): The ISCX2012 comprises diverse protocols, but the distribution of simulated attacks may not align with real-world statistics. Additionally, its absence of HTTPS traffic reduces its current relevance (Shiravi et al., 2012).

ADFA (University of New south wales 2013): ADFA includes raw call traces during regular host operations and attacks, yet it may suffer from homogeneity and lack of distinction between attack and normal behaviors (Xie and Hu, 2013; Xie et al., 2014).

CIC-IDS2017 (Canadian Institute for Cybersecurity (CIC), University of New Brunswick 2017): Offering abstract behaviors of

Table 1
NTLFlowLyzer features.

Behavior	Num.	*Feature name*	Is new	*Num.*	Feature name	Is new
Time-based	*F1*	Duration		*F13*	Packets IAT Min	
	F2	Active Max		*F14*	Packets IAT Sum	
	F3	Active Mean		*F15*	Fwd Packets IAT Max	
	F4	Active STD		*F16*	Fwd Packets IAT Mean	
	F5	Active Min		*F17*	Fwd Packets IAT STD	
	F6	Idle Max		*F18*	Fwd Packets IAT Min	
	F7	Idle Mean		*F19*	Fwd Packets IAT Sum	
	F8	Idle STD		*F20*	Bwd Packets IAT Max	
	F9	Idle Min		*F21*	Bwd Packets IAT Mean	
	F10	Packets IAT Max		*F22*	Bwd Packets IAT STD	
	F11	Packets IAT Mean		*F23*	Bwd Packets IAT Min	
	F12	Packets IAT STD		*F24*	Bwd Packets IAT Sum	
Rate-based	*F25*	Down Up Rate		*F30*	Fwd Bytes Rate	✓
	F26	Packets Rate		*F31*	Bwd Bytes Rate	✓
	F27	Fwd Packets Rate		*F32*	Packets Count	✓
	F28	Bwd Packets Rate		*F33*	Fwd Packets Count	
	F29	Bytes Rate		*F34*	Bwd Packets Count	
Header-based	*F35*	Header Bytes Max	✓	*F44*	Fwd Header Bytes Total	
	F36	Header Bytes Mean	✓	*F45*	Bwd Header Bytes Max	✓
	F37	Header Bytes STD	✓	*F46*	Bwd Header Bytes Mean	✓
	F38	Header Bytes Min	✓	*F47*	Bwd Header Bytes STD	✓
	F39	Header Bytes Total	✓	*F48*	Bwd Header Bytes Min	✓
	F40	Fwd Header Bytes Max	✓	*F49*	Bwd Header Bytes Total	✓
	F41	Fwd Header Bytes Mean	✓	*F50*	Avg Segment Size	✓
	F42	Fwd Header Bytes STD	✓	*F51*	Fwd Avg Segment Size	
	F43	Fwd Header BytesMin	✓	*F52*	Bwd Avg Segment Size	
Payload-based	*F53*	Payload Bytes Max		*F62*	Fwd Payload Bytes Min	
	F54	Payload Bytes Mean		*F63*	Fwd Payload Bytes Total	
	F55	Payload Bytes STD		*F64*	Fwd Payload Bytes Variance	✓
	F56	Payload Bytes Min		*F65*	Bwd Payload Bytes Max	
	F57	Payload Bytes Total	✓	*F66*	Bwd Payload Bytes Mean	
	F58	Payload Bytes Variance		*F67*	Bwd Payload Bytes STD	
	F59	Fwd Payload Bytes Max		*F68*	Bwd Payload Bytes Min	
	F60	Fwd Payload Bytes Mean		*F69*	Bwd Payload Bytes Total	
	F61	Fwd Payload Bytes STD		*F70*	Bwd Payload Bytes Variance	✓
Flag-based	*F71*	FIN Flag Counts		*F83*	Fwd ACK Flag Counts	✓
	F72	SYN Flag Counts		*F84*	Fwd URG Flag Counts	✓
	F73	RST Flag Counts		*F85*	Fwd ECE Flag Counts	✓
	F74	PSH Flag Counts		*F86*	Fwd CWR Flag Counts	✓
	F75	ACK Flag Counts		*F87*	Bwd FIN Flag Counts	✓
	F76	URG Flag Counts		*F88*	Bwd SYN Flag Counts	✓
	F77	ECE Flag Counts		*F89*	Bwd RST Flag Counts	✓
	F78	CWR Flag Counts		*F90*	Bwd PSH Flag Counts	✓
	F79	Fwd FIN Flag Counts	✓	*F91*	Bwd ACK Flag Counts	✓
	F80	Fwd SYN Flag Counts	✓	*F92*	Bwd URG Flag Counts	✓
	F81	Fwd RST Flag Counts	✓	*F93*	Bwd ECE Flag Counts	✓
	F82	Fwd PSH Flag Counts	✓	*F94*	Bwd CWR Flag Counts	✓
Side-based	*F95*	Avg Fwd Bytes Per Bulk		*F105*	Bwd Bulk State Count	✓
	F96	Avg Fwd Packets Per Bulk		*F106*	Bwd Bulk Size Total	✓
	F97	Avg Fwd Bulk Rate		*F107*	Bwd Bulk Packet Count	✓
	F98	Avg Bwd Bytes Per Bulk		*F108*	Bwd Bulk Duration	✓
	F99	Avg Bwd Packets Per Bulk		*F109*	Subflow Fwd Packets	
	F100	Avg Bwd Bulk Rate		*F110*	Subflow Bwd Packets	
	F101	Fwd Bulk State Count	✓	*F111*	Subflow Fwd Bytes	
	F102	Fwd Bulk Size Total	✓	*F112*	Subflow Bwd Bytes	
	F103	Fwd Bulk Packet Count	✓	*F113*	Fwd Init Win Bytes	
	F104	Fwd Bulk Duration	✓	*F114*	Bwd Init Win Bytes	

users based on various protocols, CIC-IDS2017 includes a comprehensive range of attacks, features, and criteria for reliable evaluation (Sharafaldin et al., 2018, 2019; Kaur et al., 2020).

After analyzing the datasets, we found that the CIC-IDS2017 dataset is particularly suitable for evaluating our proposed profiling model due to its comprehensive and up-to-date representation of real-world network traffic. The dataset includes diverse background network traffic data and updated attacks. Furthermore, it provides raw data in the form of PCAP files. To extract flows and features from the raw data, we utilized our analyzer, NTLFlowLyzer. Using our tool to generate our own analyzed data in CSV format, we addressed the issues and shortcomings of the CICFlowMeter CSV file provided with the dataset.

5.2. Flow extraction and CSV generation using NTLFlowLyzer

In this section, we delve into the intricacies of flow extraction and the generation of CSV files employing NTLFlowLyzer. Our analysis uncovered significant disparities between the NTLFlowLyzer-generated CSV file and the CICFlowMeter CSV file provided as part of the dataset. These disparities are of utmost importance as they impact the accuracy and reliability of the data used in our research.

One notable disparity revolves around the flow counts associated with different labels in both CSV files, as meticulously documented in Table 2. These discrepancies in flow counts are indicative of fundamental differences in how these flows are captured and recorded. It is crucial to highlight that these discrepancies are not mere data

Table 2
Number of flows per activity comparison between CSV files generated by CICFlowMeter and NTLFlowLyzer.

Category	Activity	CICFlow.	NLFlow.
Benign	Monday_Benign	529,918	495,338
	Tuesday_Benign	432,074	395,976
	Wednesday_Benign	440,031	397,053
	Thursday_Benign	168,186	133,770
	Friday_Benign	414,322	364,102
DoS	DoS_GoldenEye	10,293	8364
	DoS_Hulk	231,073	349,240
	DoS_Slowhttp	5499	6860
	DoS_Slowloris	5796	5177
	DDoS_LOIT	128,027	95,733
Brute Force	FTP_Patator	7938	9531
	SSH_Patator	5897	5949
	Web_Brute_Force	1507	2734
Botnet	Botnet_ARES	1966	5508
Port Scanning	Portscan	158,930	161,323
Vulnerability Exploitation	SQL_Injection	21	24
	Web_XSS	652	1358
	Heartbleed	11	12

anomalies but rather stem from variations in flow creation and termination processes, coupled with certain implementation and labeling inconsistencies within the CIC-IDS2017 dataset itself.

Furthermore, we observed discrepancies in feature extraction between the NTLFlowLyzer-generated CSV file and the CICFlowMeter CSV file, as depicted in Table 1. These disparities underscore the importance of a robust and consistent approach to feature extraction, a critical aspect of our research.

In summary, the utilization of NTLFlowLyzer has provided us with a more dependable and comprehensive dataset for our research endeavors. The ensuing section is dedicated to leveraging the BCCC-CIC-IDS2017 dataset (BCCC-Dataset, 2024), generated through NTLFlowLyzer, to conduct a rigorous evaluation of our proposed profiling model. The insights and findings from this evaluation are poised to contribute significantly to the advancement of intrusion detection methodologies and network security research as a whole.

6. Experiment results

This section presents experimental results from our model for profiling diverse network activities. We offer insights into experiment metrics, profiling scenarios, feature selection, behavior similarity, and pattern extraction. These findings showcase raw experiment outputs. In the following section, we will analyze and interpret these results comprehensively.

6.1. Experiment metrics

Our focus is on developing a profiling system, not a detection system. Our experiment procedure centers on "profiling evaluation", assessing the successful creation of behavioral profiles for each activity. The experiment criteria include:

- ***Correctness***: The initial step, accurately profiling all activity records, lays the foundation for subsequent enhancements.
- ***Comprehensiveness***: Ability to accurately capture the behavior of unseen records, assessed using data not utilized during profile creation.
- ***Definitiveness***: Ensuring specificity, avoiding assignment to records from other activities. A definitive profile captures only unique behaviors.

Evaluating profiles based on these criteria gauges the effectiveness of our proposed system. Refining profiles for high correctness, comprehensiveness, and definitiveness follows, leading to robust profiling.

6.2. Profiling scenarios

To comprehensively assess profile effectiveness and achieve defined goals (correctness, comprehensiveness, and definitiveness), we employed two profiling scenarios.

The first scenario adopted a multi-layer approach, creating two profiles per activity. These profiles' results were combined for the final outcome. Conversely, the second scenario followed a single-profile-per-activity approach. However, the second scenario's profile features equaled the sum of the first scenario's. For instance, the first scenario entailed dual profiles, each with four features per activity. In contrast, the second scenario comprised a single profile containing eight features per activity.

Notably, the nature of the feature selection algorithm introduces a crucial consideration with increased feature count. Features optimal for a four-feature profile might not be suitable for an eight-feature profile. In the next subsection, we delve deeper into feature selection results, providing insights into the implications of our chosen profiling scenarios.

6.3. Feature selection

For pertinent network activity features, our algorithm ran separately for each activity. Table 3 outlines chosen features for each activity and profiling scenario. To ascertain optimal feature selection and behavior similarity, three correlation algorithms (one linear, two non-linear) were employed. Behavior similarity results are detailed in the next subsection.

Our feature selection algorithm might yield multiple feature sets per activity. To enhance profile distinctiveness and definitiveness, we opted for the least common set among activities. This ensures unique features per profile, elevating definitiveness.

Fig. 6 visually shows feature correlation across activities. Node connections reflect correlation magnitude. No connection suggests a correlation below the 0.1 thresholds.

To enhance profile distinctiveness, we addressed correlations in over 30% activities. This bolsters uniqueness and definitiveness. Edges with absolute weight under 0.3 were excluded, retaining key features. This streamlines representative feature sets, boosting analysis efficiency and accuracy.

6.4. Behavior similarity

To gauge network activity similarity, we employed our behavior similarity algorithm. We executed this algorithm using three correlation techniques: Pearson, Spearman, and KendallTau. Notably, this study emphasizes similarity to exemplar activities (Benign and DDoS_Slowloris) due to space limitations. Fig. 7 illustrates these specific comparisons, aiding in recognizing activity similarity and associated risks.

6.5. Profile creation: Pattern extraction

This section outlines steps for profile creation via pattern extraction from our model. To ensure balanced training and profile data, we randomly selected 70% of activity data. Subsequent subsections detail pivotal training steps: range calculation, parameter tuning, and pattern extraction.

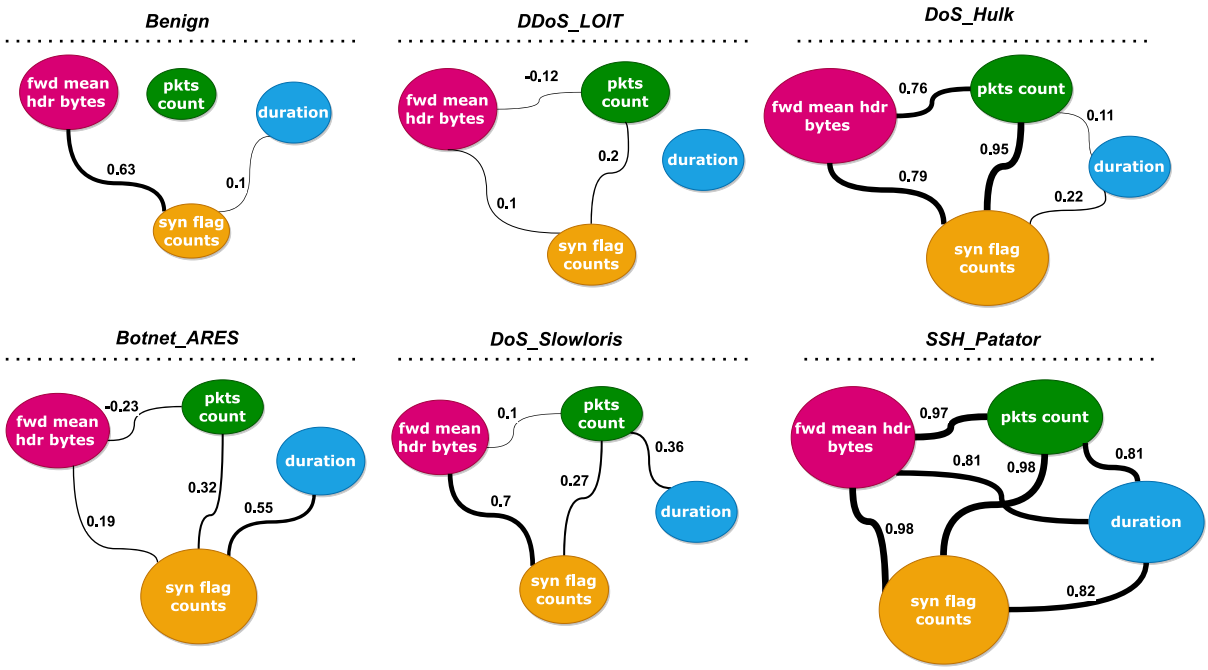


Fig. 6. Features correlations in different activities.

Table 3
Selected feature for each activity in each profile.

Activity	*1st Profile*	*2nd Profile*	*3rd Profile*
Benign	{F13, F25, F42, F41}	{F50, F52, F53, F88}	{F13, F25, F41, F42, F43, F53, F88, F114}
DDoS LOIT	{F44, F49, F52, F87}	{F32, F34, F75, F91}	{F32, F33, F42, F44, F49, F52, F69, F87}
DoS GoldenEye	{F7, F54, F71, F88}	{F14, F23, F24, F54}	{F7, F21, F41, F52, F54, F71, F88, F114}
DoS Hulk	{F11, F80, F90, F91}	{F14, F15, F19, F33}	{F11, F43, F45, F53, F80, F87, F90, F91}
DoS Slowhttp	{F13, F25, F46, F88}	{F33, F50, F66, F82}	{F13, F14, F25, F35, F41, F46, F65, F88}
DoS Slowloris	{F40, F88, F89, F97}	{F32, F110, F111, F112}	{F22, F32, F40, F81, F83, F88, F89, F97}
Botnet ARES	{F21, F31, F71, F88}	{F16, F18, F19, F33}	{F21, F25, F31, F33, F36, F71, F73, F88}
FTP Patator	{F90, F91, F111, F114}	{F32, F110, F111, F112}	{F47, F64, F80, F90, F91, F110, F111, F114}
SSH Patator	{F80, F82, F111, F112}	{F32, F110, F111, F112}	{F21, F32, F52, F80, F82, F110, F111, F112}
Port Scan	{F47, F64, F80, F110}	{F22, F32, F109, F110}	{F39, F44, F47, F64, F75, F79, F80, F110}
Web Brute Force	{F25, F32, F39, F57}	{F15, F18, F19, F35}	{F1, F19, F25, F32, F39, F57, F79, F80}
Web XSS	{F17, F30, F109, F114}	{F18, F19, F109, F110}	{F1, F17, F30, F40, F54, F109, F111, F114}

6.5.1. Range calculation

Range calculation computed Kernel Density Estimation (KDE) for each feature in each activity. The Gaussian kernel was used, revealing varied feature ranges across activities (Fig. 8). Different features demonstrate distinct behaviors in diverse activities, underlining feature selection importance. Space limits allow one feature example, but a similar analysis is applied to others.

Range calculation transforms continuous data into density representations, enhancing data distribution understanding. Identifying ranges crucially aids subsequent pattern extraction, establishing correlations among feature values per activity.

6.5.2. Pattern extraction

In the final step, profiles were constructed for each label using features and range from previous steps. Pattern extraction employed FP-Growth for association rule mining and Particle Swarm Optimization for parameter tuning (minimum support and confidence ratios). After thorough experimentation, 0.1 and 0.4 were chosen as minimum support and confidence ratios, respectively. These values ensured relevant and accurate patterns for activities.

To enhance readability, we graphically connected nodes to represent extracted patterns for each activity. Profiles are shaped by features' varying ranges and unique correlations. This visual approach illustrates permissible feature ranges and correlations, demonstrating each activity's behavior.

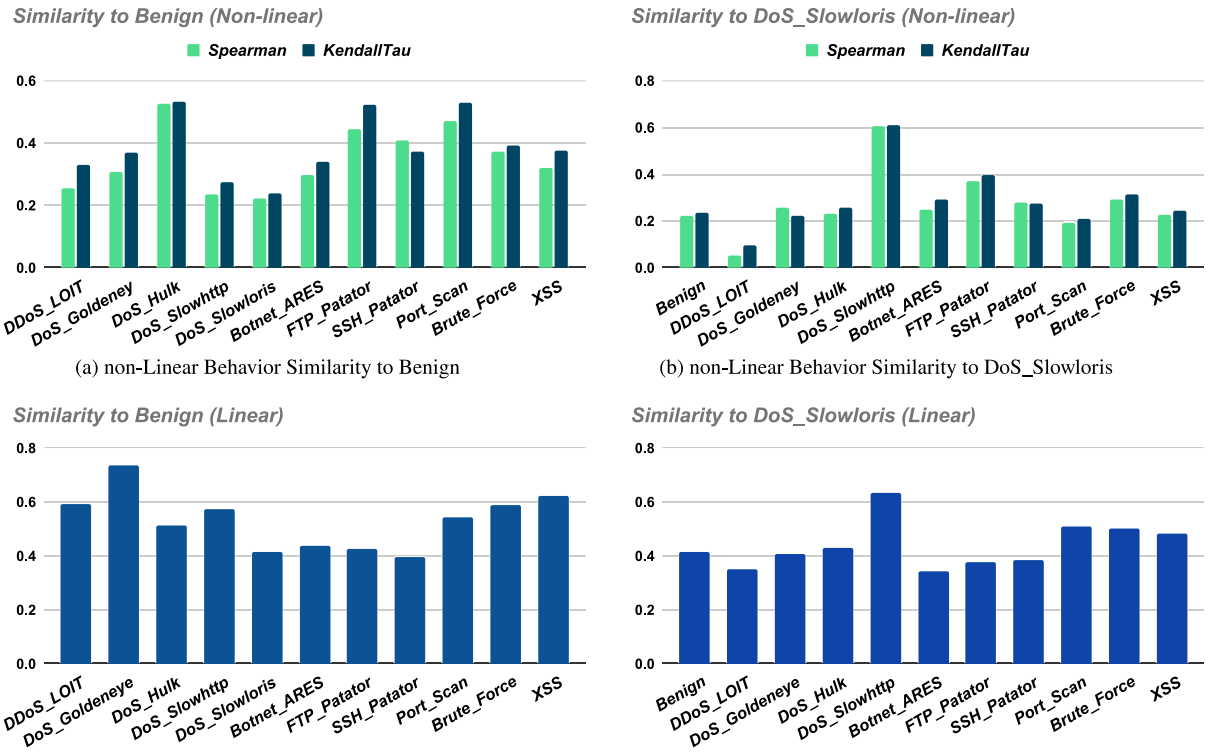
Fig. 9 showcases created profiles for some of the activities with four features. Nodes correspond to feature ranges, and edges connect based on patterns. This visualizes correlations and value possibilities, uncovering activity behavior. Though space confines us, our experiments covered all features, yielding comprehensive profiles. These profiles form the basis of our model, enabling accurate and efficient network activity profiling. It is important to highlight that due to space constraints, we have included select examples of created profiles rather than presenting all the profiles in each scenario.

6.5.3. Profile-assigning

This subsection evaluates profile correctness and comprehensiveness in both profiling scenarios, aligning with earlier discussed standards. We emphasized correctness and comprehensiveness, with definitiveness assessment reserved for future work.

Within the multi-layer approach, we explored profile output combinations via intersection and union. Results for correctness and comprehensiveness can be found in Tables 4 and 5. Diverse profiles in this multi-layer setup address complexity as features increase, necessitating intricate analysis. As shown in Tables 4 and 5, multi-profile adoption enhances system accuracy and profile comprehensiveness.

To conclude, our model exhibits promise in network activity profiling. Profile correctness and comprehensiveness are crucial. The next section deeply analyzes experimental outcomes, accounting for influencing factors. Conclusions drawn, strengths and limitations assessed,



(a) non-Linear Behavior Similarity to Benign

(b) non-Linear Behavior Similarity to DoS_Slowloris

(c) linear (Pearson) Behavior Similarity to Benign

(d) linear (Pearson) Behavior Similarity to DoS_Slowloris

Fig. 7. Behavior similarity between different activities using different correlation calculation algorithms.

Table 4
Correctness for each activity in each profile.

Activity	*1st Profile*	*2nd Profile*	*1st Profile ∩ 2nd Profile*	*1st Profile ∪ 2nd Profile*	*3rd Profile*
Benign	57.8	73.8	57.8	73.8	53.6
DDoS LOIT	47.1	79.0	47.1	79.0	28.6
DoS GoldenEye	88.4	90.0	88.4	90.0	72.9
DoS Hulk	86.2	92.7	79.0	99.9	79.0
DoS Slowhttp	81.2	84.2	75.2	90.1	75.5
DoS Slowloris	93.3	99.4	92.7	100	92.7
Botnet ARES	98.3	99.3	92.7	100	92.7
FTP Patator	99.8	99.8	99.7	100	99.7
SSH Patator	99.2	99.8	99.2	99.8	98.5
Port Scan	99.5	99.9	98.7	99.8	98.6
Web Brute Force	97.3	95.8	93.1	100	93.2
Web XSS	99.0	97.8	97.0	99.8	96.6

Table 5
Comprehensiveness for each activity in each profile.

Activity	*1st Profile*	*2nd Profile*	*1st Profile ∩ 2nd Profile*	*1st Profile ∪ 2nd Profile*	*3rd Profile*
Benign	57.8	73.8	57.8	73.9	53.5
DDoS LOIT	47.1	78.8	47.1	79.0	78.8
DoS GoldenEye	89.6	90.1	88.3	90.1	74.0
DoS Hulk	86.3	92.9	79.0	99.9	79.1
DoS Slowhttp	81.5	83.8	75.2	90.2	76.2
DoS Slowloris	92.3	99.3	92.7	100	91.6
Botnet ARES	98.4	99.2	97.6	100	95.0
FTP Patator	99.8	99.9	99.6	100	99.7
SSH Patator	99.5	99.8	99.3	99.9	99.9
Port Scan	99.6	99.9	98.6	99.9	99.6
Web Brute Force	97.4	94.2	93.2	100	91.7
Web XSS	97.8	98.3	97.1	99.8	96.0

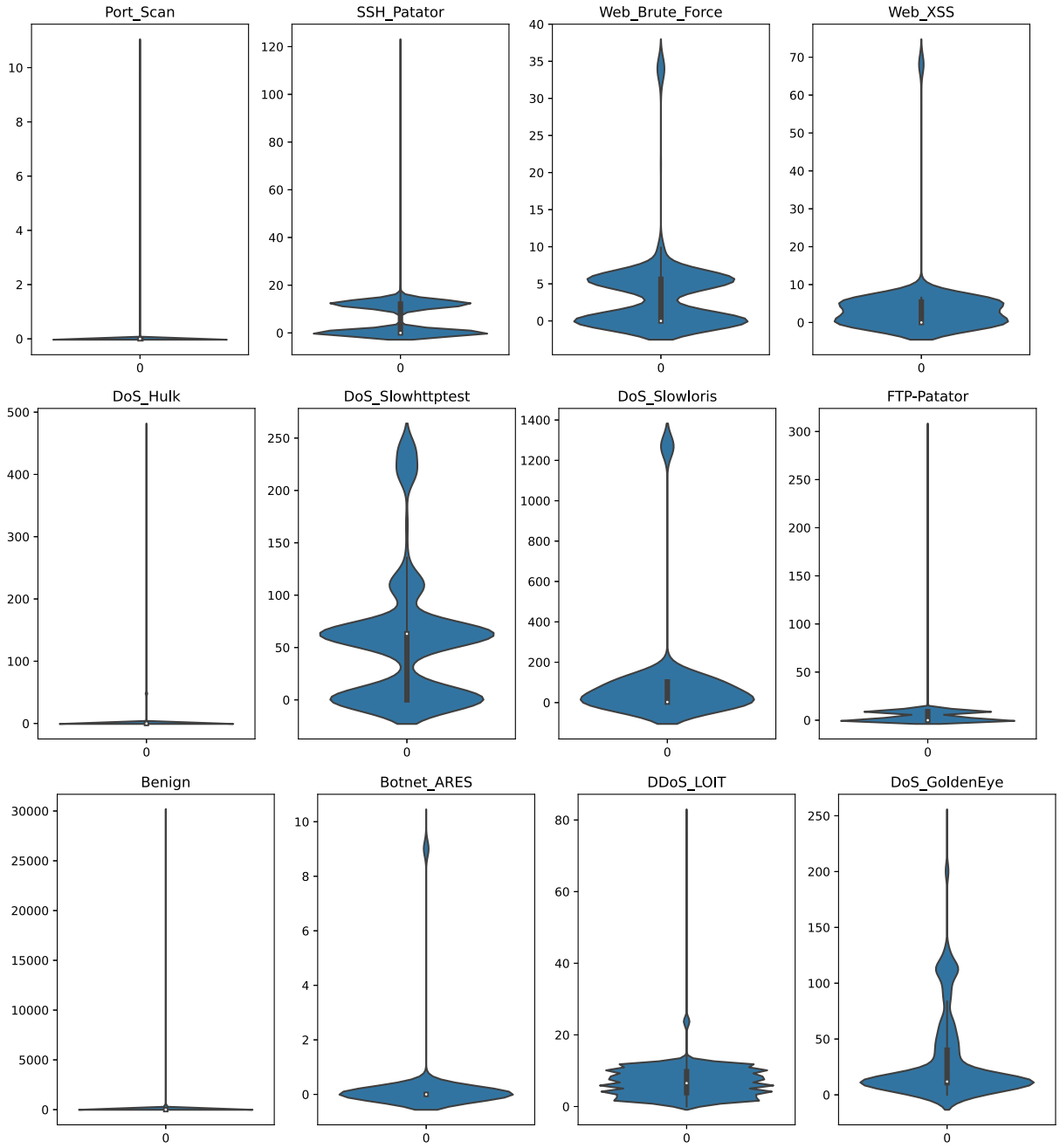


Fig. 8. Violin plot of duration values in different activities.

and future directions explored. This inclusive approach aims to offer a robust network activity profiling system.

7. Analysis, evaluation and discussion

This section conducts an in-depth analysis and discussion of our proposed behavior profiling model's foundational concepts. We explore distinct feature behavior across activities and varying feature correlations. Empirical evidence in Figs. 6 and 8 supports these ideas, affirming robust profile creation.

7.1. Idea analysis

Experimental outcomes (Fig. 8) strongly affirm distinct feature behaviors in activities, aligning with our initial hypothesis. Fig. 6 reinforces varied feature correlations across activities. These concepts synergize, yielding comprehensive, reasonable profiles validated by experiments.

Effective profiling mandates careful feature selection. Features' ranges and correlations inform this selection. Feature range drives rule extraction; correlations guide rule extraction and feature selection. Feature choice's importance during rule extraction is evident: unrelated features hinder rule identification. Absent correlations yield meaningless rules and inaccurate profiles. Relevant, correlated features are pivotal for successful rule extraction and accurate profiling. In the ensuing subsection, we elaborate on our crucial feature selection algorithm.

Our experiments substantiate our ideas and model efficacy. Merging distinct feature behaviors and correlations yields robust, comprehensive activity profiles. This distinguishes activities and furnishes network security insights, aiding anomaly detection. Model success establishes potential real-world network applications.

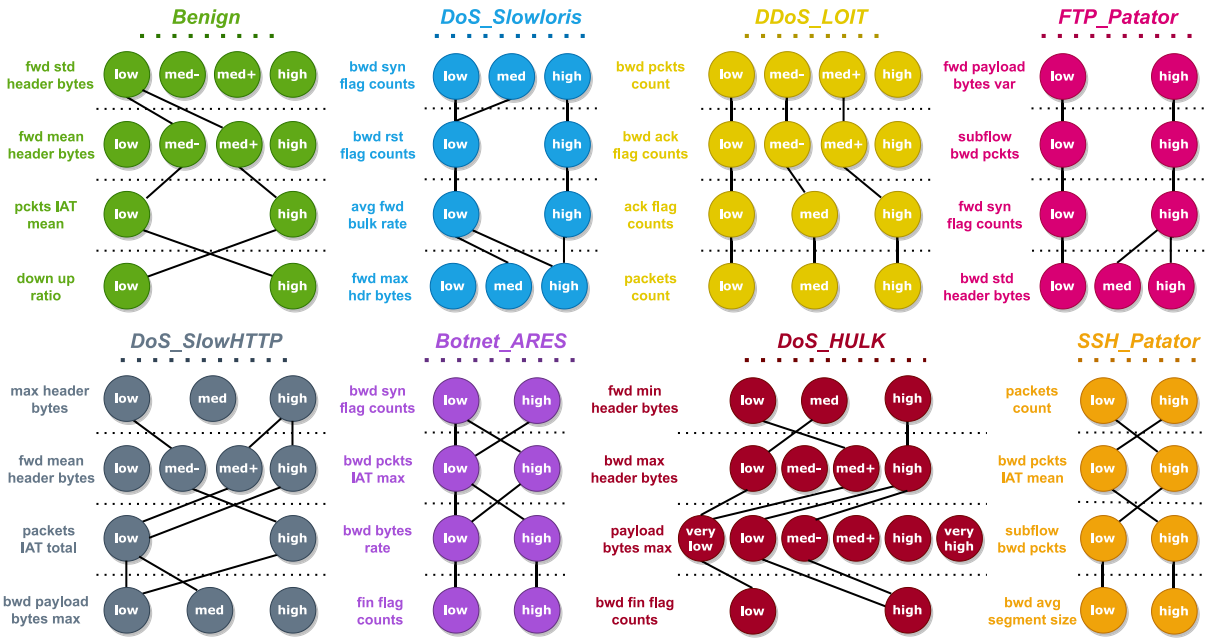


Fig. 9. Examples of created profiles for some of the activities.

7.2. Feature selection analysis

Selecting relevant features stands as a pivotal stride in robust profile creation. Our novel approach prioritizes highly correlated features, vital for uncovering inter-feature relationships within each activity. Non-correlated features yield trivial rules, spotlighting our effective selection algorithm's value. Our approach surpasses common methods like information gain. It ensures that selected features align with our profiling definition, resulting in meaningful profiles and precise behavior depiction.

Our feature selection algorithm's nature implies varying feature sets with increased feature count. For instance, profiles with four versus eight features may vary (Table 3). To address this, two scenarios were crafted. In the second, the feature count equals the first scenario's sum. Performance analysis in Section 7.6 provides an assessment of both scenarios, gauging their effectiveness and relative performance.

7.2.1. Correlation threshold analysis

Feature selection involves a defined correlation threshold to remove correlations common in over 30% of activities. A lower threshold unveils unique correlations, enhancing features and outcomes, especially for suspicious output reduction. Yet, very low thresholds might not fit limited features or extensive activities. They remove many edges, impeding robust path discovery. Higher thresholds may select common features, yielding multiple or suspicious outputs. Appropriate threshold setting is pivotal for effective feature selection.

7.2.2. Time complexity

In this section, we provide an analysis of the time complexity of our feature selection algorithm, taking into account potential optimizations through parallel computing. The optimized time complexity breakdown with parallel optimization is as follows:

1. **Initialization:** The initialization phase involves creating a empty graph with m nodes, resulting in a time complexity of $O(m)$.
2. **Loop Over Activities:** The main computational load occurs during the loop over distinct activity labels. Within this loop, we consider the following major operations:

(a) **Calculating Correlations:** The correlation calculation between each pair of features has a time complexity of $O(m(m-1)/2)$. With parallel computing, this complexity can be reduced, potentially to $O(m(m-1)/(2p))$, where p represents the degree of parallelism achieved.

(b) **Finding Strongest Path:** The process of finding the strongest path of a specified length L in a graph is computationally expensive. The time complexity for this operation is influenced by the graph's structure after edge removal. However, the integration of parallelism as an optimization strategy can significantly improve performance. By leveraging multiple processors or threads for concurrent path exploration, the time complexity can be reduced to approximately $O(m^L/p)$ in the worst case (when it is a complete graph), where p represents the number of processors or threads utilized.

The overall time complexity of the optimized algorithm with parallelization can be summarized as:

$$O\left(m + \frac{km(m-1) + 2m^L}{2p}\right) \tag{9}$$

where m represents the number of features, k denotes the number of distinct activity labels, L is the path length, and p represents the degree of parallelism.

7.3. Behavior similarity analysis

Fig. 7 reveals DoS Slowhttp as most similar to DoS Slowloris. This discovery underscores our profiling technique's reliability and effectiveness. The significant similarity between these activities attests to our ability to identify and quantify behavior patterns across activities, affirming our approach's robustness and accuracy in assessing activity similarity.

These findings hold vital implications for future behavior analysis and profiling. Recognizing behavior similarities among attack types enhances detection algorithm efficiency and effectiveness. Leveraging these similarities improves attack identification, leading to prompt, precise responses.

Remarkable consistency in most similar activity for DoS_Slowloris among three algorithms is notable. Minor similarity variations for other activities indicate the algorithm choice's secondary impact. Non-linear algorithms yielding almost identical similarity values suggest non-linearity as a general phenomenon, not critical for algorithm selection.

Finding unique correlations in DoS_Slowloris, absent in DoS_Slowhttp, is challenging. Profiles for similar activities like DoS_Slowloris and DoS_Slowhttp tend to contain more suspicious outputs. More similarity equates to a higher likelihood of detecting suspicious behavior. Further research may enhance final results for closely related activities. Overall, findings showcase behavior analysis and profiling's potency in detecting and responding to attacks.

In conclusion, our experiments underscore our profiling system's effectiveness in measuring activity similarity. A robust method for detecting and analyzing malicious activities emerges, bolstering network security and safeguarding against potential cyber threats.

7.4. Created profile analysis

Successful behavior profiling of each attack showcases our system's adeptness in characterizing attack types through extracted patterns. Let us delve into the DoS Slowloris profile (Fig. 9). This profile unveils an attacker strategy: establishing connections to a target server and sustaining them, taxing system resources.

Remarkably, our profiling system detects this by scrutinizing pivotal features like the bwd_syn_flag_counts and the bwd_rst_flag_counts. Elevated bwd_syn_flag_counts implies high bwd_rst_flag_counts (here, equal to one), signaling attacker connection initiation. Consequently, the victim responds with an "rst" packet to free occupied resources, initiating a cycle where the attacker triggers new connections, prompting victim closure, rendering resources and ports unavailable to legitimate users.

Once connected, the attacker prolongs the connection. Our system flags this behavior, too. Fig. 9 illustrates regular syn packets (established connection) alongside low "rst" flags (zero here), indicating a sustained connection to drain victim resources.

We applied analogous analysis to other profiles, affirming our system's aptitude for identifying unique attack patterns. Despite space constraints, we have consistently found our profiling system distinguishing attack patterns.

Our analysis affirms pattern identification for all profiles unique to each attack type. Profiles aptly capture corresponding activity behavior; association rule mining extracts insightful feature value relationships. Our profiling proves robust, differentiating diverse network activities. Though space limits individual profile discussion here, we have conducted similar analyses for all, yielding consistent results.

7.5. Zero-day attack profiling

Our pioneering profiling system effectively characterizes zero-day attacks through network traffic analysis. Upon encountering a new attack, the following scenarios emerge:

- **Distinct Behavior:** When no existing profiles align with the new activity, we provisionally label it "Unknown" or "Zero-day Attack" until sufficient data permits a dedicated profile creation.
- **Profile Overlap:** If the new activity shares elements with multiple profiles, refining detection involves eliminating shared patterns. Continued alignment implies malicious intent, warranting the "Unknown" or "Zero-day Attack" label until a specific profile is established.

In the realm of network security, two crucial approaches emerge, anomalous activity detection and zero-day attack detection (AlEroud and Karabatis, 2012). Abnormal activity detection is centered around spotting deviations from established network norms, essential for maintaining network integrity (Chandola et al., 2009). On the other hand, zero-day attack detection stands as a critical line of defense against threats that exploit vulnerabilities unknown to the broader security community (Ahmad et al., 2023; Hiremagalore, 2015).

In our specific approach, we have structured our system to confront the challenges posed by zero-day attacks. To achieve this, we first establish a baseline of normal behavior by meticulously analyzing the network's usual traffic patterns. This baseline acts as a reference point for what is considered typical (i.e., normal behavior) in the network environment which can be used for the abnormal activity profiling. Building on this, we create profiles for known malicious behaviors based on various attack scenarios present in our comprehensive dataset. These profiles serve as reference points for identifying attacks that have already been encountered and categorized. However, what sets our approach apart is the provision for recognizing previously unseen attacks or attack variants.

In situations where incoming network activity does not align with any of the established profiles, we provisionally label it as a "Zero-Day Attack". This classification acknowledges the unknown or novel nature of the behavior. The system can then accumulate data over time, and as it becomes more familiar with the characteristics of this unknown behavior, it can create a dedicated profile. This proactive approach ensures that emerging threats are promptly detected and addressed.

The ability to identify and respond to such novel threats is what defines our method as a zero-day attack profiling system. While we also engage in the essential practice of abnormal activity profiling to maintain network integrity, our emphasis on rapidly classifying and mitigating activities that exploit vulnerabilities unknown to the broader security community sets our approach apart, ultimately fortifying network security against emerging threats.

7.6. Performance analysis

Evaluating our profiling system's outcomes is crucial after establishing diverse profiles. Results from correctness and comprehensiveness tables (Tables 4 and 5) reveal that the combined 1st and 2nd profiles demonstrate superior overall performance across activities. This is due to these profiles analyzing behaviors from distinct perspectives, resulting in complementary coverage and improved outcomes. Moreover, both profiles proficiently detect common behaviors, evident from high correctness and comprehensiveness scores for most activities.

These findings advocate for a multi-layer profiling structure, allowing for the amalgamation of perspectives to enhance activity behavior understanding. Employing the intersection of profiles might lead to inferior results due to the diversity in perspectives and featured elements.

Considering the merging of the two profiles into one comprehensive profile presents intriguing possibilities, explored in our second scenario. Potential outcomes include the new profile mirroring the features of the first two profiles or incorporating previously absent traits. Which case emerges depends on the feature selection algorithm, without a predetermined outcome. In our experiments, the latter scenario transpired, further discussed in Table 3. Building on these cases, we conduct further analysis in the next subsection.

7.6.1. Analysis of feature count per profile

The optimal number of features necessary to profile specific behaviors has been debated. Our research challenges the efficacy of using identical feature sets across activities. Notably, distinct activities exhibit diverse feature relationships, as depicted in Fig. 6. This underscores the significance of per-profile feature selection and the individual profiling of each activity.

Exploring the ideal number of features per profile reveals insightful conclusions from our two profiling scenarios. Remarkably, utilizing four features surpasses the effectiveness of eight features across most activities. It is essential to note that increased feature usage escalates time and space demands, spanning range calculations to pattern extraction.

A notable trend is that fewer features foster stronger connections in the feature selection graph. This results in multiple feature sets of fixed lengths (e.g., four), with the best one chosen. Conversely, higher feature counts might hinder the graph's identification of connected feature sets. This suggests that sub-behaviors within activities might lack direct connections. For instance, in the Benign activity (Fig. 6), the packet count feature exhibits minimal correlation with the other three. Sub-behaviors within activities might necessitate distinct profiling, leading to potential research on sub-profiles encompassing these sub-behaviors to form a comprehensive activity profile.

Furthermore, our study reveals varying feature counts per profile for each activity. Correctness results in Table 4 highlight that Benign to DoS Slowhttp activities excel with four features, while other activities benefit from eight. This reinforces the notion of tailoring feature selection to the unique nature of each activity's behavior.

Space constraints limit our presentation to two profiling scenarios. However, future work should delve deeper into more comprehensive experiments and analysis for optimal feature counts and selection strategies. Investigating sub-behaviors and their distinct feature selection could also offer valuable research avenues.

Notably, low correctness and comprehensiveness values for specific profiles within some activities stem from limited feature availability in the evaluation, particularly for Benign. Deleting specific correlations during feature selection reduces profile effectiveness. While definitiveness was not the primary focus, it remains essential for our study. Future endeavors can explore definitiveness more exhaustively.

7.6.2. Missed attacks impact

In assessing the performance of our security system, it is crucial to delve into the impact of missed attacks, specifically focusing on the most prevalent one as indicated by the data in Tables 4 and 5. A closer examination of our results unveils an intriguing trend. Notably, attacks exhibiting a higher similarity to benign network traffic, such as DDoS LOIT, DoS HULK, DoS Slowhttp, and DoS GoldenEye, often experience a higher incidence of being missed by our system. This pattern underscores the effectiveness of the similarity calculation approach and the system's sensitivity to the subtleties in network traffic behaviors, where the proximity of an attack's behavior to that of benign activity can lead to overlooked threats.

Among these closely related attacks, DDoS LOIT stands out as the most frequently missed, warranting special attention in our analysis. The inherent challenge in detecting DDoS LOIT attacks arises from their resemblance to benign network traffic. These attacks employ tactics that closely mimic legitimate user interactions, making them intricate to differentiate from typical activity. The nuances of DDoS LOIT lie in their utilization of low-level traffic volume and a distributed sourcing approach, characteristics that superficially mirror commonplace network behaviors.

The overlooked DDoS LOIT attacks, while notable, underscore a distinctive aspect of these attacks: they tend to consume network resources without necessarily causing system downtime. The attacks may pose a resource utilization challenge, imposing an increased load on network resources and potentially leading to slower response times. However, they typically do not result in a complete system shutdown or the kind of severe service disruption associated with other types of attacks.

Moving forward, we are committed to refining our profiling system to address this specific challenge, ensuring comprehensive protection against DDoS LOIT attacks and other sophisticated threats. This iterative process will involve the incorporation of additional anomaly detection techniques, improved traffic analysis, and the development of specific profiles to detect and mitigate these types of attacks effectively.

7.7. Evaluation: Comparison with previous works

To gauge the innovation and efficacy of our proposed model, we conduct a comparative assessment with two prominent recent studies (Rabbani et al., 2020) and (Hou et al., 2022). This evaluation illuminates the strides and contributions of our approach, spanning dataset utilization, evaluation metrics, and profiling strategies. It is noteworthy that direct result comparisons with previous 'profiling' works are constrained by the utilization of different datasets in their respective studies. Consequently, our focus in this work centers on a comprehensive comparison of the fundamental aspects, theoretical foundations, and underlying philosophies of prior research endeavors, as elaborated in the Literature Review Synthesis (Section 2.2.2).

7.7.1. Data

Our comparison begins with an evaluation of data. (Rabbani et al., 2020) employed the UNSW-NB15 dataset (Moustafa and Slay, 2015), and (Hou et al., 2022) combined the CTU-13 (Garcia et al., 2014) and ISCX2012 (Shiravi et al., 2012) datasets. In contrast, we harnessed raw traffic data from CIC-IDS2017 (Lashkari et al., 2017), employing an analyzer (BCCC-NTLFlowLyzer, 2024) to generate essential CSV files. Let us examine these datasets through distinct lenses:

- **Up-to-date Relevance**: The data currency is pivotal in intrusion detection, given the continuous evolution of malicious activities. Timely training data is essential to detect novel malicious behaviors effectively. Our chosen dataset, BCCC-CIC-IDS2017 (BCCC-Dataset, 2024), boasts superior up-to-date relevance compared to other datasets. While ISCX2012 hails from 2012, CTU-13 from 2013, and UNSW-NB15 from 2015, the CIC-IDS2017 dataset stems from 2017.
- **Dataset Magnitude**: Dataset size profoundly impacts profile comprehensiveness and model assessment. ISCX2012 contains 133,450, CTU-13 comprises 180,896, and UNSW-NB15 comprises 1,964,509 flows. In contrast, our BCCC-CIC-IDS2017 dataset (BCCC-Dataset, 2024) incorporates 2,438,052 flows (as detailed in Table 2). This dataset's larger scale furnishes richer analytical opportunities and robust profiling.
- **Activity Diversity**: The number of activities (labels) bears significance on several fronts. It underscores profile precision, complexity of testing, and resemblance to real-world scenarios where diverse attacks target systems. ISCX2012 encompasses 6 activities, CTU-13 has 14, UNSW-NB15 includes 10, whereas our dataset comprises 14 distinct activities. Our dataset captures diverse activities, reflecting real-world complexity.

In summation, our profiling approach excels in dataset aspects, leveraging an up-to-date, larger, and more comprehensive dataset featuring diverse activities. These attributes substantiate the potency and relevance of our proposed model.

7.7.2. Experiment metrics

A notable distinction between our proposed model and prior studies lies in the evaluation metrics employed. Preceding works primarily concentrated on detection, striving to attain optimal detection rates for recognized activities. In contrast, our central focus rested on profiling, aiming to formulate comprehensive behavioral profiles for each activity. This pivotal differentiation engenders fundamental disparities in approach and objectives between these two model categories.

Within a detection-centric framework, the core objective involves discerning if a given activity corresponds to established activities or signifies a potential new attack (zero-day attack). This approach hinges heavily on the specific training dataset, rendering detection outcomes susceptible to shifts in data distribution or emergent attack types. Consequently, detection models might not exhibit robust generalizability to novel or unexplored data.

Conversely, our profiling-centric approach strives to create inclusive behavior profiles for each activity. These profiles encapsulate distinct

activity patterns and attributes, impervious to the particular profiling dataset. Thus, the profiling model attains greater resilience, applicable across diverse datasets and scenarios, so long as activity behavior remains consistent.

Prioritizing profiling over detection distinguishes our model as a pioneering advancement in network security. We establish a systematic and comprehensive framework for behavior profiling, offering insightful comprehension of individual activity behavior. This enriched perception of activity patterns markedly enhances overall network security system efficacy, facilitating heightened threat identification, response, and adaptation to novel attack typologies.

To summarize, our model's divergence in evaluation metrics and the primacy of profiling heralds a substantive evolution from earlier endeavors, constituting a notable stride in behavior-centric network security.

7.7.3. Profiling solution

An integral facet of our comparative analysis involves the comprehensive nature of our proposed profiling solution. Diverging from prior approaches fixated on the profiling model per se, we formulated an end-to-end solution for behavior profiling. This holistic solution enacts crucial steps, commencing with raw network traffic data and culminating in the generation of individual behavioral profiles. The pivotal constituents of our profiling solution encompass:

This study presents a comprehensive and all-encompassing profiling solution, encompassing vital components that span raw network traffic data to individual activity behavioral profile construction. Our approach encompasses a series of pivotal stages: network analysis, flow and feature extraction, feature selection, pattern extraction, and culminating in behavioral profile formulation. Distinguished from prior works centered solely on the profiling model, we acknowledge the equal significance of each constituent within the overarching profiling system.

The crafting of a robust profiling system mandates meticulous attention to each phase. Every element assumes a cardinal role in influencing the caliber and precision of the final profiles. For example, data quality directly reverberates onto profiling outcomes; compromised data can skew profiles and engender unreliable findings. Additionally, precise flow delineation and accurate labeling are indispensable for capturing genuine activity behavior.

Furthermore, the quantity and caliber of selected features are paramount for yielding precise and informative profiles. Insufficient features might inadequately represent multifaceted behaviors across activities, leading to incomplete or erroneous profiles. Correspondingly, inaccuracies in feature implementation or flawed code can propagate errors throughout the profiling process.

Our solution accentuates the critical significance of feature selection and pattern extraction, determinants that profoundly impact the profiling system's efficacy. Via a novel feature selection algorithm and harnessing FP-Growth and PSO for pattern extraction, we achieve more exact and distinctive activity profiles. These profiles encapsulate idiosyncratic behavior, fortifying our solution's resilience and adaptability across diverse datasets and scenarios.

However, a limitation of our study pertains to the reduced emphasis on profile definitiveness. The dataset's plethora of labels yielded fewer unique feature correlations within the feature selection algorithm. Consequently, fewer features were available for profile creation, diminishing profile definitiveness. Mitigating this challenge and augmenting the definitiveness metric represent fertile areas for future enhancement and inquiry.

In summation, our comparative analysis underscores the strides and enhancements inherent in our comprehensive profiling solution, juxtaposed with earlier approaches. Distinguished by multifaceted evaluation metrics, our approach deviates from earlier models that exclusively fixated on the profiling model. This comparative analysis furnishes empirical substantiation, validating the efficacy and ascendancy of our proposed solution, and highlighting its distinctiveness and contributions within the realm of behavioral profiling.

8. Conclusion and future prospects

This paper introduces a sophisticated and adaptable solution for creating precise behavioral profiles tailored to diverse network activities. Our approach tackles profile creation challenges across the entire process, from raw data handling to the NTLFlowLyzer analyzer, feature extraction, implementation, dataset preparation (BCCC-CIC-IDS2017 (BCCC-Dataset, 2024)), feature selection, behavior similarity assessment, range calculation, pattern extraction, and profile generation. Our profiling solution applicability empowers organizations, including firewalls, IDS/IPS, Security Information and Event Management (SIEM), Unified Threat Management (UTM) systems, and others, to strengthen their network security and threat detection mechanisms, particularly in the context of detecting zero-day and previously unknown malicious activities.

A robust profiling system hinges on high-quality data. Extensive dataset analysis revealed raw and analyzed data limitations. While addressing raw data issues requires new data generation, this goes beyond this study's scope. Therefore, we utilized the most reliable dataset's raw data. Additionally, we designed and implemented the NTLFlowLyzer analyzer to enhance the analyzed data generation (CSV files), validated against CICFlowMeter.

Our framework centers on two key principles: feature behavior distinctiveness and varying feature correlations across activities. Extensive experimentation validates these principles. Our novel graph-based feature selection algorithm guarantees relevant feature selection, as demonstrated in experiments and analysis. Furthermore, based on the feature selection graph, we introduce Behavior Similarity, quantifying activity similarity, which is valuable for identifying critical and complex attacks, regardless of conditions. Our pattern extraction phase captures behavior intricacies, making our profiling core adept at representing network behavior nuances. Our solution's advancements hold profound implications for behavior analysis and profiling research, particularly in swiftly identifying malicious activities. We underscore non-linearity's significance in behavior patterns, offering the potential to refine profiling algorithms and address zero-day threats.

Our evaluation framework surpasses conventional correctness and comprehensiveness, highlighting our system's feature quantity and quality advantage. This approach prompts deeper consideration of evaluation metrics' interplay, zero-day profiling, and selected feature set robustness. Our proposed framework effectively profiles eight malicious activities with over 99.8% correctness and comprehensiveness. It also successfully profiles other activities promisingly. These outcomes and insights from our experiments offer valuable guidance for accurate behavioral profiling.

Our solution can significantly enhance network security and threat detection systems, including detecting complex zero-day attacks. Comparative analysis with previous works demonstrates our framework's exceptional performance, reinforcing its uniqueness and practicality. Our insights into selected feature quantity and quality strengthen our dedication to refining field approaches and cultivating resilient behavior analysis systems.

In the future, research avenues beckon. We prioritize the third metric – definitiveness- – in refining our system. Ensembles hold the potential to bolster robustness and precision. Exploring diverse, larger-scale datasets promises a comprehensive view of our system's performance.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: MohammadMoein Shaf reports financial support was provided by York University.

Acknowledgments

The authors acknowledge the grant from Canada Research Chair - Tier II (#CRC-2021-00340), and the Natural Sciences and Engineering Research Council of Canada — NSERC (#RGPIN-2020-04701) — to Arash Habibi Lashkari.

Data availability

The source code for NTLFlowLyzer is publicly available on GitHub (BCCC-NTLFlowLyzer, 2024) and the data for BCCC-CIC-IDS2017 Dataset is publicly available on the BCCC website (BCCC-Dataset, 2024).

References

Abdi, H., 2007. The Kendall rank correlation coefficient. In: Encyclopedia of Measurement and Statistics. Sage, Thousand Oaks, CA, pp. 508–510.

Abdulganiyu, O.H., Ait Tchakoucht, T., Saheed, Y.K., 2023. A systematic literature review for network intrusion detection system (ids). International journal of information security 22 (5), 1125–1162.

Afzal, Z., Lindskog, S., 2016. Ids rule management made easy. In: 2016 8th International Conference on Electronics, Computers and Artificial Intelligence. ECAI, IEEE, pp. 1–8.

Ahmad, R., Alsmadi, I., Alhamdani, W., Tawalbeh, L., 2023. Zero-day attack detection: a systematic literature review. Artif. Intell. Rev. 1–79.

Akhiat, Y., Asnaoui, Y., Chahhou, M., Zinedine, A., 2021. A new graph feature selection approach. In: 2020 6th IEEE Congress on Information Science and Technology. CiSt, IEEE, pp. 156–161.

Al Jallad, K., Aljnidi, M., Desouki, M.S., 2020. Anomaly detection optimization using big data and deep learning to reduce false-positive. J. Big Data 7 (1), 1–12.

AlEroud, A., Karabatis, G., 2012. A contextual anomaly detection approach to discover zero-day attacks. In: 2012 International Conference on Cyber Security. IEEE, pp. 40–45.

Aljanabi, M., Ismail, M.A., Ali, A.H., 2021. Intrusion detection systems, issues, challenges, and needs. Int. J. Comput. Intell. Syst. 14 (1), 560–571.

Alrawashdeh, K., Purdy, C., 2016. Toward an online anomaly intrusion detection system based on deep learning. In: 2016 15th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 195–200.

AlYousef, M.Y., Abdelmajeed, N.T., 2019. Dynamically detecting security threats and updating a signature-based intrusion detection system's database. Procedia Comput. Sci. 159, 1507–1516.

Asif, M., Abbas, S., Khan, M., Fatima, A., Khan, M.A., Lee, S.-W., 2021. MapReduce based intelligent model for intrusion detection using machine learning technique. J. King Saud Univ. Comput. Inf. Sci.

Ayyagari, M.R., Kesswani, N., Kumar, M., Kumar, K., 2021. Intrusion detection techniques in network environment: a systematic review. Wirel. Netw. 27 (2), 1269–1285.

Baldini, G., Amerini, I., 2022. Online Distributed Denial of Service (DDoS) intrusion detection based on adaptive sliding window and morphological fractal dimension. Comput. Netw. 210, 108923.

Barros, P.H., Chagas, E.T., Oliveira, L.B., Queiroz, F., Ramos, H.S., 2022. Malware-SMELL: A zero-shot learning strategy for detecting zero-day vulnerabilities. Comput. Secur. 120, 102785.

BCCC-Dataset, 2024. BCCC updated intrusion detection dataset 2017 (BCCC-CIC-ids2017). Behaviour-Centric Cybersecurity Center (BCCC) URL https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/.

BCCC-NTLFlowLyzer, 2024. Network and transport layers flow analyzer (ntlFlowLyzer), Retrieved 10 April 2023. Behaviour-Centric Cybersecurity Center (BCCC) URL https://github.com/ahlashkari/NTLFlowLyzer.

Bolboaca, S.D., Jäntschi, L., 2006. Pearson versus Spearman, Kendall's tau correlation analysis on structure-activity relationships of biologic active compounds. Leonardo J. Sci. 5 (9), 179–200.

Brown, C., Cowperthwaite, A., Hijazi, A., Somayaji, A., 2009. Analysis of the 1999 darpa/lincoln laboratory ids evaluation data with netadhict. In: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications. IEEE, pp. 1–7.

Chandola, V., Banerjee, A., Kumar, V., 2009. Anomaly detection: A survey. ACM Comput. Surv. (CSUR) 41 (3), 1–58.

Chen, Y.C., 2017. A tutorial on kernel density estimation and recent advances. Biostatist. Epidemiol. 1 (1), 161–187.

Chitrakar, R., Huang, C., 2012. Anomaly based intrusion detection using hybrid learning approach of combining k-medoids clustering and naive bayes classification. In: 2012 8th International Conference on Wireless Communications, Networking and Mobile Computing. IEEE, pp. 1–5.

Chok, N.S., 2010. Pearson's Versus Spearman's and Kendall's Correlation Coefficients for Continuous Data (Ph.D. thesis). University of Pittsburgh.

Cohen, I., Huang, Y., Chen, J., Benesty, J., Benesty, J., Chen, J., Huang, Y., Cohen, I., 2009. Pearson correlation coefficient. Noise Reduct. Speech Process. 1–4.

Cui, Z., Gao, X.-Z., Deb, S., 2019. Theory and applications of soft computing methods. Neural Comput. Appl. 31, 1983–1985.

Dina, A.S., Manivannan, D., 2021. Intrusion detection based on machine learning techniques in computer networks. Internet Things 16, 100462.

Fürnkranz, J., 2013. Rule-based methods. In: Dubitzky, W., Wolkenhauer, O., Cho, K.-H., Yokota, H. (Eds.), Encyclopedia of Systems Biology. Springer New York, New York, NY, pp. 1883–1888. http://dx.doi.org/10.1007/978-1-4419-9863-7_610.

Garcia, N., Alcaniz, T., González-Vidal, A., Bernabe, J.B., Rivera, D., Skarmeta, A., 2021. Distributed real-time SlowDoS attacks detection over encrypted traffic using Artificial Intelligence. J. Netw. Comput. Appl. 173, 102871.

Garcia, S., Grill, M., Stiborek, J., Zunino, A., 2014. An empirical comparison of botnet detection methods. Comput. Secur. 45, 100–123.

Guo, Y., 2022. A review of Machine Learning-based zero-day attack detection: Challenges and future directions. Comput. Commun.

He, X., Dai, H., Ning, P., Dutta, R., 2015. Dynamic IDS configuration in the presence of intruder type uncertainty. In: 2015 IEEE Global Communications Conference. GLOBECOM, IEEE, pp. 1–6.

Herrera-Semenets, V., Hernández-León, R., van den Berg, J., 2022. A fast instance reduction algorithm for intrusion detection scenarios. Comput. Electr. Eng. 101, 107963.

Herrmann, D., Banse, C., Federrath, H., 2013. Behavior-based tracking: Exploiting characteristic patterns in DNS traffic. Comput. Secur. 39, 17–33.

Hiremagalore, S., 2015. Zero-Day Attack Detection Using Collaborative and Transduction-Based Anomaly Detectors (Ph.D. thesis). George Mason University.

Hou, J., Liu, F., Lu, H., Tan, Z., Zhuang, X., Tian, Z., 2022. A novel flow-vector generation approach for malicious traffic detection. J. Parallel Distrib. Comput. 169, 72–86.

Hsu, Y.F., He, Z., Tarutani, Y., Matsuoka, M., 2019. Toward an online network intrusion detection system based on ensemble learning. In: 2019 IEEE 12th International Conference on Cloud Computing. CLOUD, IEEE, pp. 174–178.

Ibrahim, D., 2016. An overview of soft computing. Procedia Comput. Sci. 102, 34–38.

Imran, M., Haider, N., Shoaib, M., Razzak, I., et al., 2022. An intelligent and efficient network intrusion detection system using deep learning. Comput. Electr. Eng. 99, 107764.

Janiesch, C., Zschech, P., Heinrich, K., 2021. Machine learning and deep learning. Electr. Mark. 31 (3), 685–695.

Jensi, R., Jiji, G.W., 2016. An enhanced particle swarm optimization with levy flight for global optimization. Appl. Soft Comput. 43, 248–261.

Kachitvichyanukul, V., 2012. Comparison of three evolutionary algorithms: GA, PSO, and DE. Ind. Eng. Manag. Syst. 11 (3), 215–223.

Kapetanakis, S., Filippoupolitis, A., Loukas, G., Al Murayziq, T.S., 2014. Profiling Cyber Attackers Using Case-Based Reasoning. CEUR.

Kaur, G., Habibi Lashkari, A., Rahali, A., 2020. Intrusion traffic detection and characterization using deep image learning. In: IEEE Intl Conf on Cyber Science and Technology Congress. CyberSciTech, pp. 55–62. http://dx.doi.org/10.1109/DASC-PICom-CBDCom-CyberSciTech49142.2020.00025.

Keerthi, S.S., Lin, C.J., 2003. Asymptotic behaviors of support vector machines with Gaussian kernel. Neural Comput. 15 (7), 1667–1689.

Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: Proceedings of ICNN'95-International Conference on Neural Networks, Vol. 4. IEEE, pp. 1942–1948.

Khan, M.A., Karim, M.R., Kim, Y., 2019. A scalable and hybrid intrusion detection system based on the convolutional-LSTM network. Symmetry 11 (4), 583.

Khan, K., Sahai, A., 2012. A comparison of BA, GA, PSO, BP and LM for training feed forward neural networks in e-learning context. Int. J. Intell. Syst. Appl. 4 (7), 23.

Khoshraftar, S., An, A., 2022. A survey on graph representation learning methods. arXiv preprint arXiv:2204.01855.

Khraisat, A., Gondal, I., Vamplew, P., Kamruzzaman, J., 2019. Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity 2 (1), 1–22.

Kim, T., Pak, W., 2022. Real-time network intrusion detection using deferred decision and hybrid classifier. Future Gener. Comput. Syst. 132, 51–66.

Kocher, G., Kumar, G., 2021. Machine learning and deep learning methods for intrusion detection systems: recent developments and challenges. Soft Comput. 25 (15), 9731–9763.

Kolascyk, E., 2013. Statistical analysis of network data. In: SAMSI Program on Complex Networks. Boston university.

Kotsiantis, S., Kanellopoulos, D., 2006. Association rules mining: A recent overview. GESTS Int. Trans. Comput. Sci. Eng. 32 (1), 71–82.

Lashkari, A.H., Gil, G.D., Mamun, M.S.I., Ghorbani, A.A., 2017. Characterization of tor traffic using time based features. In: International Conference on Information Systems Security and Privacy. 2, SciTePress, pp. 253–262.

Li, W., Tug, S., Meng, W., Wang, Y., 2019. Designing collaborative blockchained signature-based intrusion detection in IoT environments. Future Gener. Comput. Syst. 96, 481–489.

Li, B., Wang, Y., Xu, K., Cheng, L., Qin, Z., 2022. DFAID: Density-aware and feature-deviated active intrusion detection over network traffic streams. Comput. Secur. 118, 102719.

Liang, H., Wu, J., Mumtaz, S., Li, J., Lin, X., Wen, M., 2019. MBID: Micro-blockchain-based geographical dynamic intrusion detection for V2X. IEEE Commun. Mag. 57 (10), 77–83.
Liao, H.J., Lin, C.H.R., Lin, Y.C., Tung, K.Y., 2013. Intrusion detection system: A comprehensive review. J. Netw. Comput. Appl. 36 (1), 16–24.
Lin, P., Ye, K., Xu, C.Z., 2019. Dynamic network anomaly detection system by using deep learning techniques. In: Cloud Computing–CLOUD 2019: 12th International Conference, Held As Part of the Services Conference Federation, SCF 2019, San Diego, CA, USA, June 25–30, 2019, Proceedings 12. Springer, pp. 161–176.
Liu, Q., Wang, D., Jia, Y., Luo, S., Wang, C., 2022. A multi-task based deep learning approach for intrusion detection. Knowl.-Based Syst. 238, 107852.
Luna, J.M., Fournier-Viger, P., Ventura, S., 2019. Frequent itemset mining: A 25 years review. Wiley Interdiscip. Rev. Data Min. Knowl. Discov. 9 (6), e1329.
Marron, J.S., Wand, M.P., 1992. Exact mean integrated squared error. Ann. Statist. 20 (2), 712–736.
McHugh, J., 2000. Testing intrusion detection systems: a critique of the 1998 and 1999 darpa intrusion detection system evaluations as performed by lincoln laboratory. ACM Trans. Inf. Syst. Secur. 3 (4), 262–294.
Meng, G., Saddeh, H., 2020. Applications of machine learning and soft computing techniques in real world. Int. J. Comput. Appl. Inf. Technol. 12 (1), 298–302.
Midway, S.R., 2020. Principles of effective data visualization. Patterns 1 (9).
Mighan, S.N., Kahani, M., 2021. A novel scalable intrusion detection system based on deep learning. Int. J. Inf. Secur. 20, 387–403.
Monzer, M.-H., Beydoun, K., Ghaith, A., Flaus, J.M., 2022. Model-based IDS design for ICSs. Reliab. Eng. Syst. Saf. 108571.
Moustafa, N., Slay, J., 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference. MilCIS, IEEE, pp. 1–6.
Muraleedharan, N., Parmar, A., Kumar, M., 2010. A flow based anomaly detection system using chi-square technique. In: 2010 IEEE 2nd International Advance Computing Conference. IACC, IEEE, pp. 285–289.
Mushtaq, E., Zameer, A., Umer, M., Abbasi, A.A., 2022. A two-stage intrusion detection system with auto-encoder and LSTMs. Appl. Soft Comput. 121, 108768.
Myers, L., Sirois, M.J., 2004. Spearman correlation coefficients, differences between. In: Encyclopedia of Statistical Sciences, vol. 12, Wiley Online Library.
Nechaev, B., Allman, M., Paxson, V., Gurtov, A., 2004. Lawrence Berkeley National Laboratory (lbnl)/icsi Enterprise Tracing Project. LBNL/ICSI, Berkeley, CA.
Nehinbe, J.O., 2009. A simple method for improving intrusion detections in corporate networks. In: International Conference on Information Security and Digital Forensics. Springer, pp. 111–122.
Poli, R., Kennedy, J., Blackwell, T., 2007. Particle swarm optimization: An overview. Swarm Intell. 1, 33–57.
Potharaju, S.P., Sreedevi, M., 2018. Correlation coefficient based candidate feature selection framework using graph construction. Gazi Univ. J. Sci. 31 (3), 775–787.
Pourhabibi, T., Ong, K.L., Kam, B.H., Boo, Y.L., 2020. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. Decis. Support Syst. 133, 113303.
Pratama, D.H., Suyanto, S., 2020. Comparison of PSO, FA, and BA for discrete optimization problems. In: 2020 3rd International Seminar on Research of Information Technology and Intelligent Systems. ISRITI, IEEE, pp. 17–20.
Prusty, S., Levine, B.N., Liberatore, M., 2011. Forensic investigation of the OneSwarm anonymous filesharing system. In: Proceedings of the 18th ACM Conference on Computer and Communications Security. pp. 201–214.
Qiu, W., Ma, Y., Chen, X., Yu, H., Chen, L., 2022. Hybrid intrusion detection system based on Dempster-Shafer evidence theory. Comput. Secur. 117, 102709.
Rabbani, M., Wang, Y.L., Khoshkangini, R., Jelodar, H., Zhao, R., Hu, P., 2020. A hybrid machine learning approach for malicious behaviour detection and recognition in cloud computing. J. Netw. Comput. Appl. 151, 102507.
Ravi, V., Chaganti, R., Alazab, M., 2022. Recurrent deep learning-based feature fusion ensemble meta-classifier approach for intelligent network intrusion detection system. Comput. Electr. Eng. 102, 108156.
Raykar, V.C., Duraiswami, R., 2006. Fast optimal bandwidth selection for kernel density estimation. In: Proceedings of the 2006 SIAM International Conference on Data Mining. SIAM, pp. 524–528.
Rodriguez, J.D., Perez, A., Lozano, J.A., 2009. Sensitivity analysis of k-fold cross validation in prediction error estimation. :IEEE Trans. Pattern Anal. Mach. Intell. 32 (3), 569–575.
Sagala, A., 2015. Automatic SNORT IDS rule generation based on honeypot log. In: 2015 7th International Conference on Information Technology and Electrical Engineering. ICITEE, IEEE, pp. 576–580.
Sangster, B., O'Connor, T., Cook, T., Fanelli, R., Dean, E., Morrell, C., Conti, G.J., 2009. Toward instrumenting network warfare competitions to generate labeled datasets. In: CSET.
Sarker, I.H., 2021. Deep learning: a comprehensive overview on techniques, taxonomy, applications and research directions. SN Comput. Sci. 2 (6), 420.
Sato, M., Yamaki, H., Takakura, H., 2012. Unknown attacks detection using feature extraction from anomaly-based ids alerts. In: 2012 IEEE/IPSJ 12th International Symposium on Applications and the Internet. IEEE, pp. 273–277.
Shafi, M., Lashkari, A.H., Mohanty, H., 2024b. Unveiling malicious dns behavior profiling and generating benchmark dataset through application layer traffic analysis. Computers and Electrical Engineering 118, 109436.
Shafi, M., Lashkari, A.H., Rodriguez, V., Nevo, R., 2024a. Toward generating a new cloud-based distributed denial of service (ddos) dataset and cloud intrusion traffic characterization. Information 15 (4), 195.
Shaikh, S.A., Chivers, H., Nobles, P., Clark, J.A., Chen, H., 2009. Towards scalable intrusion detection. Netw. Secur. 2009 (6), 12–16.
Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSp 1, 108–116.
Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2019. An evaluation framework for network security visualizations. Comput. Secur. 84, 70–92. http://dx.doi.org/10.1016/j.cose.2019.03.005, URL https://www.sciencedirect.com/science/article/pii/S0167404818308952.
Shawkat, M., Badawi, M., El-ghamrawy, S., Arnous, R., El-desoky, A., 2022. An optimized FP-growth algorithm for discovery of association rules. J. Supercomput. 1–28.
Shiravi, A., Shiravi, H., Tavallaee, M., Ghorbani, A.A., 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. Comput. Secur. 31 (3), 357–374.
Silva, J.V.V., de Oliveira, N.R., Medeiros, D.S., Lopez, M.A., Mattos, D.M., 2022. A statistical analysis of intrinsic bias of network security datasets for training machine learning mechanisms. Ann. Telecommun. 77 (7–8), 555–571.
Singh, A.K., Kumar, A., Maurya, A.K., 2014. An empirical analysis and comparison of apriori and FP-growth algorithm for frequent pattern mining. In: 2014 IEEE International Conference on Advanced Communications, Control and Computing Technologies. IEEE, pp. 1599–1602.
Singh, R., Kumar, H., Singla, R., 2015. An intrusion detection system using network traffic profiling and online sequential extreme learning machine. Expert Syst. Appl. 42 (22), 8609–8624.
Sonchack, J., Aviv, A.J., Smith, J.M., 2015. Cross-domain collaboration for improved IDS rule set selection. J. Inf. Secur. Appl. 24, 25–40.
Song, J., Takakura, H., Okabe, Y., Eto, M., Inoue, D., Nakao, K., 2011. Statistical analysis of honeypot data and building of Kyoto 2006+ dataset for NIDS evaluation. In: Proceedings of the First Workshop on Building Analysis Datasets and Gathering Experience Returns for Security. pp. 29–36.
Sperotto, A., Sadre, R., Vliet, F.v., Pras, A., 2009. A labeled data set for flow-based intrusion detection. In: International Workshop on IP Operations and Management. Springer, pp. 39–50.
Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A., 2009. A detailed analysis of the KDD cup 99 data set. In: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications. Ieee, pp. 1–6.
Tharwat, A., Schenck, W., 2021. A conceptual and practical comparison of PSO-style optimization algorithms. Expert Syst. Appl. 167, 114430.
Tomandl, A., Fuchs, K.P., Federrath, H., 2014. REST-net: A dynamic rule-based IDS for VANETs. In: 2014 7th IFIP Wireless and Mobile Networking Conference. WMNC, IEEE, pp. 1–8.
Unwin, A., 2020. Why is data visualization important? what is important in data visualization? Harvard Data Sci. Rev. 2 (1), 1.
Vellido, A., 2020. The importance of interpretability and visualization in machine learning for applications in medicine and health care. Neural Comput. Appl. 32 (24), 18069–18083.
von Ziegler, L., Sturman, O., Bohacek, J., 2021. Big behavior: challenges and opportunities in a new era of deep behavior profiling. Neuropsychopharmacology 46 (1), 33–44.
Wang, Z., Fok, K.W., Thing, V.L., 2022. Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study. Comput. Secur. 113, 102542.
Wang, Y., Meng, W., Li, W., Li, J., Liu, W.X., Xiang, Y., 2018. A fog-based privacy-preserving approach for distributed signature-based intrusion detection. J. Parallel Distrib. Comput. 122, 26–35.
Wang, C., Song, W., 2019. A modified particle swarm optimization algorithm based on velocity updating mechanism. Ain Shams Eng. J. 10 (4), 847–866.
Wang, X., Yan, Y., Ma, X., 2020. Feature selection method based on differential correlation information entropy. Neural Process. Lett. 52, 1339–1358.
Węglarczyk, S., 2018. Kernel density estimation and its application. In: ITM Web of Conferences, Vol. 23. EDP Sciences, p. 00037.
Xie, M., Hu, J., 2013. Evaluating host-based anomaly detection systems: A preliminary analysis of adfa-ld. In: 2013 6th International Congress on Image and Signal Processing, Vol. 3. CISP, IEEE, pp. 1711–1716.
Xie, M., Hu, J., Slay, J., 2014. Evaluating host-based anomaly detection systems: Application of the one-class SVM algorithm to ADFA-LD. In: 2014 11th International Conference on Fuzzy Systems and Knowledge Discovery. FSKD, IEEE, pp. 978–982.
Zeng, N., Wang, Z., Liu, W., Zhang, H., Hone, K., Liu, X., 2020. A dynamic neighborhood-based switching particle swarm optimization algorithm. IEEE Trans. Cybern. 52 (9), 9290–9301.
Zhang, Z., Hancock, E.R., 2011. A graph-based approach to feature selection. In: International Workshop on Graph-Based Representations in Pattern Recognition. Springer, pp. 205–214.
Zhang, Y., Wang, S., Ji, G., et al., 2015. A comprehensive survey on particle swarm optimization algorithm and its applications. Math. Prob. Eng. 2015.
Zhou, H., Wang, X., Zhu, R., 2022. Feature selection based on mutual information with correlation coefficient. Appl. Intell. 1–18.

MohammadMoein Shafi is a graduate student pursuing a Master's degree in Computer Science at York University. Holding a bachelor's degree in Computer Engineering from the University of Tehran, MohammadMoein has a keen interest in Cybersecurity, Computer Networks, Internet of Things (IoT), Artificial Intelligence, and Network Analysis. As a Research Assistant at the esteemed Behavior-Centric Cybersecurity Center (BCCC), he actively contributes to advancements in digital security and technology innovation.

Dr. Arash Habibi Lashkari is a Canada Research Chair (CRC) in Cybersecurity. He is a Senior member of IEEE and an Associate Professor at York University. He is the author of ten published books and more than 110 academic articles on various cybersecurity-related topics. He co-authors the national award-winning article series "Understanding Canadian Cybersecurity Laws", recently recognized with a Gold Medal at the 2020 Canadian Online Publishing Awards. Dr. Lashkari has over 25 years of teaching experience spanning several international universities, has received 15 awards at international computer security competitions – including three gold awards – and was recognized as one of Canada's Top 150 Researchers for 2017. Building on over two decades of concurrent industrial and development experience in network, software, and computer security, Dr. Lashkari's current work involves the development of vulnerability detection technology to protect network systems against cyberattacks. He simultaneously supervises multiple research and development teams working on several projects related to network traffic analysis, malware analysis, Honeynet, and threat hunting.

Arousha Haghighian Roudsari is currently working as a Research Professor at the School of Computing (Department of AI Software), Gachon University, South Korea. She obtained her Ph.D. in Industrial Engineering from Inha University in 2022. She received an M.Sc. degree in Industrial Engineering from Universiti Teknologi Malaysia in 2013 and a B.Sc. degree in Industrial Engineering from Islamic Azad University, Tehran North Branch, Iran, in 2009. Her current research interests include data mining, natural language processing, deep learning, big data, information retrieval, patent analysis, and cybersecurity.